MARSA MAROC

ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

RAPPORT *de stage*

SYSTÈME INTELLIGENT DE GESTION DU PESAGE, DE VÉRIFICATION D'IDENTITÉ ET DE SUIVI DES LIVRAISONS PORTUAIRES

4èmme Année Informatique et Réseau - Filière Développement Digital et Système d'Information

Société d'acceuil

Marsa Maroc

Encadré par :
Badr CHERKAOUI

Période du stage :
10 Aout - 20 Septembre 2026

Réalisé par :
Chaimaa LAHLOU

Année Univeristaire : 2025-2026

Remerciement

Je tiens à exprimer ma profonde gratitude à **Marsa Maroc**, l'organisme qui m'a accueillie durant cette période de stage. Cette expérience m'a offert l'opportunité de découvrir un environnement professionnel stimulant et enrichissant, propice à mon apprentissage et à mon développement personnel.

Mes remerciements les plus sincères s'adressent à mon encadrant, **M. CHERKAOUI Badr**, pour son accompagnement constant, sa disponibilité, sa patience et ses précieux conseils qui ont grandement contribué à la réussite de ce stage.

Je souhaite également remercier **l'École Marocaine de Science de l'Ingénieur** de Casablanca (EMSI), et plus particulièrement le département **Génie Informatique**, pour la qualité de la formation dispensée tout au long de mon parcours académique.

Je tiens à exprimer ma reconnaissance aux membres du jury **EL FAZAZI HANAA**

KALNA FATIMA pour l'intérêt qu'ils portent à ce travail en acceptant de l'examiner et de l'évaluer.

Enfin, je remercie l'ensemble des collaborateurs de Marsa Maroc pour leur accueil chaleureux, leur disponibilité et le partage bienveillant de leur expertise durant cette période.

Dédicace

À mes chers parents,
Pour leur amour inconditionnel, leurs sacrifices et leurs prières qui ont éclairé mon chemin.
Qu'ils trouvent dans ce modeste travail l'expression de ma gratitude et de mon affection.

À ma famille,
Pour leur soutien constant et leurs encouragements tout au long de mon parcours.

À mes amis et camarades,
Qui ont partagé avec moi les moments de doute et de réussite, et dont la présence a rendu cette aventure plus belle.

À tous ceux qui croient en moi et m'inspirent à donner le meilleur de moi-même.

Résume

Contexte et problématique

Dans le cadre de mon stage de six semaines chez Marsa Maroc, il m'a été confié la conception et le développement d'un système digital complet couvrant deux volets complémentaires de l'activité portuaire : le contrôle du pesage des camions au niveau du pont-bascule, et le suivi des livraisons de marchandises arrivant par voie maritime. Ces deux processus reposaient jusqu'ici sur des méthodes largement manuelles, peu tracées, avec un risque d'erreur et de fraude difficile à détecter a posteriori.

Périmètre global du projet

Le projet se structure en deux modules :

Module 1 — Système de pesage intelligent (Mizan), développé durant le stage. Ce module couvre l'ensemble du cycle de vie d'une pesée : identification automatique du camion par reconnaissance optique de caractères (OCR) sur les plaques du tracteur et de la remorque, authentification du chauffeur par badge QR code personnel, vérification visuelle de son identité assistée par capture photo automatique (comparée à une photo de référence par un agent humain, sans recours à une reconnaissance faciale automatisée pour des raisons de conformité réglementaire), puis enregistrement de la pesée à l'entrée (poids à vide) et à la sortie (poids en charge), avec calcul automatique du poids net et détection d'anomalies. L'ensemble des données est centralisé dans un tableau de bord web permettant aux superviseurs de suivre l'activité en temps réel.

Module 2 — Application mobile de suivi des livraisons maritimes, à réaliser. Ce second module doit permettre aux agents portuaires de saisir numériquement, depuis un smartphone ou une tablette, les informations relatives aux marchandises déchargées d'un navire pour le compte d'un client (par exemple une livraison de sucre pour Cosumar) : client concerné, quantité, type de marchandise, référence du navire. Ces données serviront de base à la facturation de la prestation de manutention par Marsa Maroc, et à terme, pourront être recoupées avec les sorties enregistrées par le module de pesage pour renforcer la traçabilité globale du flux (quantité entrée par bateau comparée à la quantité totale sortie par camions).

Architecture technique d'ensemble

Le système repose sur une architecture en couches : un backend Node.js/Express exposant une API REST sécurisée par authentification JWT, une base de données PostgreSQL centralisant camions, chauffeurs, pesées et (à terme) livraisons, un frontend web React pour le dashboard de supervision, une future application mobile pour la saisie terrain, et des scripts Python dédiés à la vision par ordinateur (lecture de plaques via OpenCV/EasyOCR et RoboFlow) et à la simulation du pont-bascule en l'absence de matériel physique sur site. La communication en temps réel (poids en direct, résultats de détection) est assurée par Socket.io.

Mots-clés : OCR , Node,js/Express , API REST , JWT , React , OpenCV , EasyOCR , RoboFlow, PostgreSQL

Abstract

Context and problem statement

As part of a six-week internship at Marsa Maroc, I was tasked with designing and developing a complete digital system covering two complementary aspects of port operations: truck weighing control at the weighbridge, and the tracking of maritime cargo deliveries. Both processes had until now relied largely on manual methods, poorly traced and prone to errors and fraud that were difficult to detect after the fact.

Overall project scope

The project is structured around two modules:

Module 1 — Mizan, an intelligent weighing system, developed during the internship. This module covers the full lifecycle of a weighing operation: automatic truck identification through Optical Character Recognition (OCR) applied to the tractor and trailer license plates, driver authentication via a personal QR code badge, identity verification assisted by automatic photo capture (compared against a reference photo by a human agent, without automated facial recognition, for regulatory compliance reasons), followed by recording the weighing at entry (empty weight) and exit (loaded weight), with automatic net weight calculation and anomaly detection. All data is centralized in a web dashboard allowing supervisors to monitor activity in real time.

Module 2 — A mobile application for tracking maritime deliveries, yet to be developed. This second module is intended to let port agents digitally record, from a smartphone or tablet, information about cargo unloaded from a ship on behalf of a client (for example, a sugar delivery for Cosumar): client concerned, quantity, cargo type, vessel reference. This data will serve as the basis for billing Marsa Maroc's handling service, and could eventually be cross-checked against the exits recorded by the weighing module to strengthen overall flow traceability (quantity received by ship compared to the total quantity leaving by truck).

Overall technical architecture

The system relies on a layered architecture: a Node.js/Express backend exposing a REST API secured by JWT authentication, a PostgreSQL database centralizing trucks, drivers, weighings and (eventually) deliveries, a React web frontend for the supervision dashboard, a future mobile application for field data entry, and Python scripts dedicated to computer vision (license plate reading via OpenCV/EasyOCR , RoboFlow) and weighbridge simulation in the absence of on-site physical hardware. Real-time communication (live weight, detection results) is handled via Socket.io.

Keywords : OCR , Node,js/Express , API REST , JWT , React , OpenCV , EasyOCR , RoboFlow , PostgreSQL

Table des matières

1.1 Introduction....16
1.2 Présentation de l'organisme d'accueil....16
1.2.1 Présentation de Marsa Maroc....16
1.2.2 Historique de Marsa Maroc....18
1.2.3 Missions de Marsa Maroc....19
1.2.4 Organisation du port de Casablanca....21
1.2.5 Division Système d'information....23
1.2 Cahier de Charge....26
1.2.1 Contexte du projet....26
1.2.2 Problématique....26
1.2.3 Vue d'ensemble du projet....27
1.2.4 Objectif général....27
1.2.5 Principe général de fonctionnement....28
1.2.6 Modules fonctionnels de la solution....29
1.2.6.1 Besoins fonctionnels....29
1.2.6.2 Besoins non fonctionnels....31
1.2.7 Technologies de développement....32
1.2.8 Technologies liées à l'intelligence artificielle et à l'OCR....33
1.2.9 Architecture fonctionnelle de la solution....33
Conclusion....35
3.1 Modélisation UML....37
3.1.1 Diagramme de cas d'utilisation....37
3.1.3 Diagramme de Classe....40
3.1.4 Diagramme de Séquence....43
3.1.5 Diagramme d'activité....45
3.1.6 Diagramme d'etat transition....48
3.1.7 Diagramme de deploiement....50
3.2 Conception de la base de données....52
3.2.1 Choix du modèle relationnel....52
3.2.2 Modèle conceptuel de données....53
3.2.3 Modèle logique de données....54

3.2.4 Description des principales tables 54
a. Table utilisateurs 54
b. Table chauffeurs 55
c. Table camions 55
d. Table prestations 55
e. Table prestation_autorisations 56
f. Table pesees 56
g. Table tickets 57
h. Table marchandises 57
3.2.5 Clés étrangères et relations 58
3.2.6 Contraintes de la base de données 59
b. Contrainte de non-nullité 59
3.2.7 Intégrité des données 60
3.2.8 Schéma simplifié des relations 61
3.2.9 Conclusion 62
3.3 Conception des API 63
3.3.1 Architecture générale des API 63
3.3.2 Principales routes de l'API 63
3.3.3 Conclusion de la conception des API 65
3.4 Conclusion 65
4.1 Introduction 68
4.2 Environnement de développement 68
4.2.1 Antigravity IDE 68
4.2.2 Node.js 69
4.2.3 npm 69
4.2.4 PostgreSQL 69
4.2.5 Git et GitHub 70
4.2.8 Postman 70
4.2.9 Roboflow 70
4.2.10 Expo 70
4.2.11 Docker 71
4.3 Organisation du backend 71

4.4 Organisation du frontend web 73
4.5 Implémentation du backend 73
4.5.1 Configuration du serveur 73
4.5.2 Connexion à PostgreSQL 74
4.5.3 Authentification 74
4.5 Implémentation du frontend web 74
4.5.1 Page de connexion 75
4.5.2 Tableau de bord 75
4.5.3 Gestion des chauffeurs 76
4.5.4 Écran de pesée 77
4.5.5 Identification OCR 77
4.5.6 Consultation des tickets 78
4.6 Intégration du service OCR 79
4.6.1 Présentation du service OCR 79
4.6.2 Traitement des erreurs OCR 79
4.7 Fonctionnement Roboflow 79
4.7.1 Traitement de Workflow 79
4.7.2 Capture et envoi de l'image 80
4.7.3 Détection de la plaque 81
4.7.4 Recherche des contours et correction de perspective 81
4.7.6 Amélioration de l'image 82
4.7.8 Lecture des groupes numériques 82
4.7.9 Reconnaissance de la lettre arabe 83
4.7.10 Problèmes de reconnaissance OCR 85
Conclusion 86
5.1 Introduction 88
5.2 Objectifs 88
5.3 Contraintes 88
5.3.1 Contraintes matérielles 88
5.3.2 Contraintes réseau 89
5.3.3 Contraintes d'utilisation 89
5.3.4 Contraintes de sécurité 89

5.4 Choix technologiques 89
5.5 Architecture mobile 90
5.5.1 Communication REST 90
5.5.2 Communication temps réel 91
5.6 Organisation des écrans 91
5. 7 Navigation 92
5.8 Maquettes 92
5.9 Diagrammes mobiles 93
5.9.1 Diagramme de cas d'utilisation 93
5.9.2 Diagramme de séquence 94
5.9.3 Diagramme d'activité 95
5.10 Conclusion 96
6.1 Introduction 98
6.2 Stratégie de test 98
6.3 Tests fonctionnels 98
6.4 Tests API 99
6.5 Tests Web 100
6.6 Tests PDA 100
6.7 Tests de performance 100
6.8 Validation métier 100
6.9 Déploiement 101
6.10 Limites 102
6.11 Conclusion 102

Liste des Figures

Figure 1 Logo Marsa Maroc 16
Figure 2 Vue du terminal à conteneurs de Marsa Maroc au port de Casablanca. 17
Figure 3 Frise chronologique de Marsa Maroc 19
Figure 4 – Opérations de manutention des conteneurs au sein d'un terminal exploité par 20
Figure 5 Navire porte-conteneurs en escale dans un terminal exploité par Marsa Maroc. 20
Figure 6 Équipements portuaires modernes utilisés par Marsa Maroc pour l'exploitation des terminaux 21
Figure 7 Vue aérienne du Port de Casablanca.....22
Figure 8 Plan des terminaux du Port de Casablanca. 23
Figure 9 Architecture fonctionnelle de la solution 34
Figure 10 Diagramme de Cas d'Utilisation 38
Figure 11 Diagramme de Classes.....40
Figure 12 Diagramme de Séquence 43
Figure 13 Diagramme d'activté 46
Figure 14 Diagramme d'état-transition.....48
Figure 15 Diagramme de Deploiement 50
Figure 16 Tableaux db part 1.....61
Figure 17 Tableaux db part 2.....62
Figure 18 Antigravity 68
Figure 19 Node.js.....69
Figure 20 npm 69
Figure 21 PostgreSQL.....69
Figure 22 Github 70
Figure 23 Postman 70
Figure 24 Roboflow.....70
Figure 25 Expo Go 71
Figure 26 Organisation Backend 72
Figure 27 Squeelize 74
Figure 28 Page Connexion 75
Figure 29 Tableau de bord.....76
Figure 30 Infos Chauffeur 76
Figure 31 Ecran Pessage.....77
Figure 32 Detection Matricule 77
Figure 33 Ticket d'Entrée 78
Figure 34 Fonctionnement Roboflow.....80
Figure 35 Preview Roboflow pour Remorque 83
Figure 36 Preveiw Roboflow Tracteur pt1 84

Figure 37 Preview Roboflow pour Tracteur pt2 85
Figure 38 Traitement Roboflow 85
Figure 39 Diagramme Cas d'Utilisation PDA 93
Figure 40 Diagramme de Sequence PDA 94
Figure 41 Diagramme d'activité PDA 95
Figure 42 Diagramme de Gantt 102

Liste des Tableaux

Tableau 1Chiffres clés de Marsa Maroc (2024) 18
Tableau 2 Modèle logique de la base de données 54
Tableau 3 Les principales clés étrangères 58
Tableau 4 Principales routes de l'API 63
Tableau 5 Tableau de technologie de PDA 89
Tableau 6 Tests Fonctionnels 98

Tableau d’abreviation

Abréviation	Signification
API	Application Programming Interface
DB	Database
DBMS / SGBD	Système de Gestion de Base de Données
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IP	Internet Protocol
OCR	Optical Character Recognition
PDA	Personal Digital Assistant
QR	Quick Response
REST	Representational State Transfer
SQL	Structured Query Language
UML	Unified Modeling Language
UI	User Interface
UX	User Experience
API REST	API basée sur l'architecture REST
JSON	JavaScript Object Notation
PDF	Portable Document Format
TCP/IP	Transmission Control Protocol / Internet Protocol
Docker	Plateforme de conteneurisation
ORM	Object-Relational Mapping
JWT	JSON Web Token
IDE	Integrated Development Environment

Introduction générale

La transformation numérique des processus industriels et logistiques constitue aujourd'hui un enjeu important pour les entreprises. L'utilisation de solutions informatiques permet notamment d'améliorer la traçabilité des opérations, de réduire les tâches manuelles et de faciliter l'accès aux informations nécessaires à la prise de décision.

Dans ce contexte s'inscrit le projet **MIZAN – Gate de Pesage**, réalisé au sein de Marsa Maroc. Le projet porte sur la conception et le développement d'une solution numérique destinée à accompagner la gestion des opérations effectuées au niveau d'une gate de pesage. Le processus de pesée nécessite en effet la coordination de plusieurs informations, notamment l'identification du véhicule, l'identification du chauffeur, la prestation concernée, le contrôle des autorisations, l'enregistrement des poids et la génération des documents associés.

L'objectif principal du projet est de proposer une solution permettant de centraliser et de digitaliser ces différentes opérations. MIZAN repose ainsi sur une architecture composée de plusieurs éléments complémentaires : une application Web destinée à la supervision, une application mobile PDA destinée aux opérations sur le terrain, un backend chargé du traitement des données et des règles métier, ainsi qu'une base de données PostgreSQL. Des technologies complémentaires telles que le QR Code, l'OCR et la communication temps réel sont également intégrées afin d'automatiser certaines étapes du processus.

La réalisation de cette solution nécessite plusieurs étapes, allant de l'analyse des besoins jusqu'aux tests et à la validation. La première partie du travail consiste à étudier le contexte existant, à identifier les acteurs et à définir les besoins fonctionnels et non fonctionnels. Une phase de conception permet ensuite de définir l'architecture du système, le modèle de données, les API et les différents diagrammes UML.

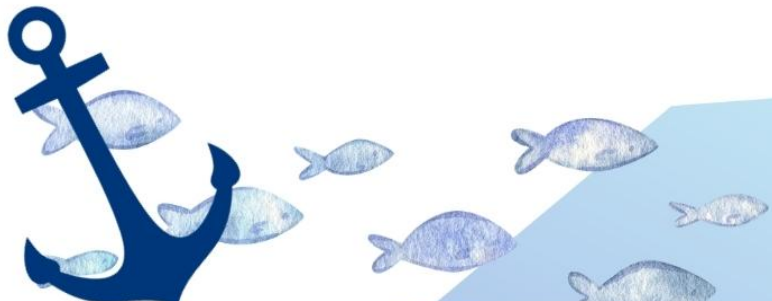
La phase de réalisation porte ensuite sur le développement du backend, de l'application Web et de l'application mobile PDA, ainsi que sur l'intégration du service OCR et des mécanismes de communication entre les différents composants. Une attention particulière est également accordée à la gestion des autorisations, à la traçabilité des opérations et à la gestion des erreurs.

Enfin, des tests fonctionnels, techniques et métier sont réalisés afin de vérifier la cohérence du système et son adéquation avec les besoins identifiés.



Chapitre 1

CONTEXTE DE PROJET





1.1 Introduction

Avant d'entrer dans le détail technique de ce projet, il est nécessaire de poser le cadre dans lequel il a été mené. Ce premier chapitre s'attache donc à présenter l'environnement du stage, à commencer par l'organisme d'accueil, Marsa Maroc, dont l'activité et le fonctionement conditionnent directement la nature des besoins auxquels ce projet devait répondre. Une fois ce cadre posé, le chapitre revient sur la problématique à l'origine du projet, en explicitant les limites concrètes de la situation existante ainsi que les enjeux qui en découlent pour l'entreprise. Il précise enfin les objectifs fixés en collaboration avec mon encadrant, ainsi que le cahier des charges qui a guidé l'ensemble des choix techniques présentés dans la suite de ce rapport. Ces éléments constituent le point de départ indispensable pour comprendre les décisions méthodologiques détaillées au chapitre suivant.

1.2 Présentation de l'organisme d'accueil

1.2.1 Présentation de Marsa Maroc

Figure 1 Logo Marsa Maroc

Marsa Maroc est le principal opérateur portuaire du Royaume du Maroc. Créée en 2006 dans le cadre de la réforme du secteur portuaire sous la dénomination Société d'Exploitation des Ports (SODEP), l'entreprise exerce aujourd'hui sous le nom de Marsa Maroc S.A.. Constituée sous la forme d'une société anonyme, elle est spécialisée dans l'exploitation des terminaux portuaires et la manutention des marchandises. Son siège social est situé à Casablanca. Aujourd'hui filiale du Groupe Tanger Med, Marsa Maroc exploite 25 terminaux répartis sur 11 ports à travers le Royaume, ainsi qu'une présence internationale via une gestion déléguée au port de Cotonou, au Bénin. L'entreprise se positionne comme le 6e plus grand opérateur à conteneurs en Afrique, un statut qui reflète l'ampleur de ses activités sur le continent.

Depuis son intégration au Groupe Tanger Med, Marsa Maroc poursuit le développement de ses activités à l'échelle nationale et internationale. En 2024, l'entreprise exploite 25 terminaux, répartis dans les principaux ports commerciaux du Maroc, et assure également une présence internationale grâce à la gestion des terminaux 1 et 5 du port de Cotonou (Bénin). Grâce à son expertise et à la qualité de ses services, Marsa Maroc occupe une position de leader sur le marché portuaire marocain



Figure 2 Vue du terminal à conteneurs de Marsa Maroc au port de Casablanca.

L'exercice 2024 a marqué une année record pour le groupe. Le chiffre d'affaires consolidé a dépassé, pour la première fois, 5 milliards de dirhams, soutenu par un trafic global de 63,3 millions de tonnes de marchandises manutentionnées et un trafic conteneurisé de près de 2,9 millions d'EVP (Équivalents Vingt Pieds). Ces performances témoignent de l'importance des flux logistiques traités quotidiennement dans les terminaux exploités par Marsa Maroc et mettent en évidence les enjeux liés à l'optimisation et à l'automatisation des opérations portuaires.

À la fin de l'année 2024, Marsa Maroc comptait 2 154 collaborateurs, mobilisés autour de valeurs d'engagement, de performance, de transparence et d'innovation, afin d'offrir des services portuaires répondant aux standards internationaux les plus exigeants.

INDICATEUR	VALEUR (2024)
DATE DE CREATION	2006
STATUT JURIDIQUE	Société Anonyme (S.A.)
SIEGE SOCIAL	Casablanca, Maroc
ACTIONNAIRE DE REFERENCE	Groupe Tanger Med
TERMINAUX EXPLOITES	25
PRESENCE GEOGRAPHIQUE	10 ports au Maroc + Port de Cotonou (Bénin)
CHIFFRE D'AFFAIRES	Plus de 5 milliards de dirhams
TRAFIC MANUTENTIONNE	63,3 millions de tonnes
TRAFIC CONTENEURISE	2,9 millions d'EVP
EFFECTIF	2154 collaborateurs

Tableau 1Chiffres clés de Marsa Maroc (2024)

1.2.2 Historique de Marsa Maroc

Avant la réforme du secteur portuaire marocain, la gestion et l'exploitation des ports de Royaume étaient assurées par l'Office d'Exploitation des Ports (ODEP), établissement public chargé de l'ensemble des activités portuaires. Afin d'améliorer la compétitivité du secteur et de moderniser sa gouvernance, la loi n°15-02 relative aux ports a instauré une profonde réforme du système portuaire marocain. Cette réforme a conduit, en 2006, à la création de deux entités distinctes : l'Agence Nationale des Ports (ANP), chargée des missions de régulation et d'autorité portuaire, et la Société d'Exploitation des Ports (SODEP), responsable de l'exploitation des terminaux portuaires. Depuis 2007, le SODEP exerce ses activités sous la marque commerciale Marsa Maroc, devenue aujourd'hui la référence nationale dans le domaine de l'exploitation portuaire.

Au fil des années, Marsa Maroc a consolidé sa position grâce à l'obtention de nouvelles concessions portuaires, au développement de ses infrastructures et à la création de plusieurs filiales spécialisées. L'introduction de la société à la Bourse de Casablanca en 2016 a marqué une étape importante de son développement, en renforçant sa gouvernance et ses capacités d'investissement. Plus récemment, l'entreprise a poursuivi son expansion à l'échelle internationale à travers la gestion des terminaux 1 et 5 du port de Cotonou au Bénin, tout en obtenant la concession du Terminal à Conteneurs Est du

port de Nador West Med, illustrant ainsi sa stratégie de croissance et son ambition de devenir un acteur portuaire de référence en Afrique.



Figure 3 Frise chronologique de Marsa Maroc

1.2.3 Missions de Marsa Maroc

En tant que principal opérateur portuaire du Royaume, Marsa Maroc a pour mission d'assurer l'exploitation, le développement et la gestion des terminaux portuaires, tout en garantissant des services répondant aux standards internationaux en matière de qualité, de sécurité et de performance. L'entreprise met son expertise au service des importateurs, des exportateurs et des compagnies maritimes en proposant une offre complète de services portuaires et logistiques.

1. Exploitation des terminaux portuaires L'exploitation des terminaux constitue la mission principale de Marsa Maroc. L'entreprise assure la gestion opérationnelle des terminaux qui lui sont concédés, en coordonnant l'ensemble des opérations nécessaires au traitement des navires et des marchandises. Elle veille à optimiser l'utilisation des infrastructures portuaires, à réduire les temps d'escale des navires et à garantir une exploitation efficace des équipements. Cette mission contribue directement à améliorer la fluidité des échanges commerciaux ainsi que la compétitivité des ports marocains.

2. Manutention des marchandises et des conteneurs Marsa Maroc assure les opérations de manutention des marchandises depuis leur arrivée au port jusqu'à leur chargement ou leur enlèvement. Ces opérations comprennent le chargement, le déchargement, le stockage et le transfert des marchandises à l'aide d'équipements spécialisés. L'entreprise traite différents types de trafic, notamment les conteneurs, les marchandises diverses, les vracs solides, les vracs liquides ainsi que les véhicules roulants (Ro-Ro).

Grâce à des équipements modernes et à un personnel qualifié, Marsa Maroc garantit des opérations rapides, sécurisées et conformes aux normes internationales.



Figure 4 – Opérations de manutention des conteneurs au sein d'un terminal exploité par

3. Prestations de services portuaires et logistiques En complément des opérations de manutention, Marsa Maroc propose un ensemble de services destinés à accompagner les compagnies maritimes et les opérateurs économiques. Selon les ports et les concessions exploités, ces prestations comprennent notamment les services liés aux escales des navires, la gestion des espaces de stockage ainsi que différentes solutions logistiques adaptées aux besoins des clients. L'objectif est d'assurer une gestion efficace des flux de marchandises tout en garantissant un niveau élevé de qualité de service.



Figure 5 Navire porte-conteneurs en escale dans un terminal exploité par Marsa Maroc.

4. Modernisation et innovation des infrastructures portuaires Dans une démarche d'amélioration continue, Marsa Maroc investit régulièrement dans la modernisation de ses infrastructures et de ses équipements. L'entreprise déploie des portiques de dernière génération, des engins de manutention performants ainsi que des solutions numériques destinées à optimiser les opérations portuaires. Elle accorde également une place importante à la transformation digitale et à l'automatisation des processus afin d'améliorer la productivité, la sécurité et la qualité des services proposés.



Figure 6 Équipements portuaires modernes utilisés par Marsa Maroc pour l'exploitation des terminaux

5. Contribution au développement du commerce national et international par son activité, Marsa Maroc joue un rôle essentiel dans le développement économique du Royaume. L'entreprise facilite les échanges commerciaux entre le Maroc et ses partenaires internationaux en assurant un traitement performant des marchandises dans les principaux ports du pays. Son développement récent à l'international, notamment à travers la gestion de terminaux au port de Cotonou au Bénin, illustre sa volonté de renforcer sa présence en Afrique et de contribuer au développement des plateformes logistiques régionales.

1.2.4 Organisation du port de Casablanca

Le Port de Casablanca est le premier port de commerce du Royaume du Maroc et constitue une plateforme logistique stratégique pour les échanges maritimes nationaux et internationaux. Situé sur la façade atlantique, il joue un rôle majeur dans le traitement des flux de marchandises, en assurant le transit d'une grande diversité de trafics tels que les conteneurs, les vracs solides, les vracs liquides, les marchandises diverses et le trafic roulier (Ro-Ro).

Afin de répondre aux exigences des différents opérateurs économiques, le Port de Casablanca dispose d'infrastructures portuaires modernes permettant d'assurer efficacement les opérations de réception, de manutention, de stockage et d'expédition

des marchandises. Ces infrastructures sont organisées en plusieurs terminaux spécialisés, chacun étant dédié à un type de trafic particulier.

Le fonctionnement du Port de Casablanca repose sur la coordination de plusieurs acteurs institutionnels et économiques, notamment l'Agence Nationale des Ports (ANP), en tant qu'autorité portuaire et organisme de régulation, les concessionnaires chargés de l'exploitation des terminaux, dont Marsa Maroc, ainsi que les administrations de contrôle, les compagnies maritimes, les transitaires et les différents prestataires participant à la chaîne logistique portuaire.



Figure 7 Vue aérienne du Port de Casablanca.

Organisation des terminaux

> Afin de répondre aux exigences des différents trafics maritimes, le Port de Casablanca est organisé en plusieurs terminaux spécialisés, chacun étant conçu pour assurer le traitement d'un type particulier de marchandises. Cette spécialisation permet d'améliorer l'efficacité des opérations portuaires, d'optimiser l'utilisation des infrastructures et de réduire les délais de traitement des navires.

— Terminal à conteneurs Il est dédié au traitement des marchandises transportées dans des conteneurs normalisés. Ce terminal dispose de portiques de quai, d'engins de manutention et de vastes aires de stockage permettant d'assurer les opérations de chargement, de déchargement et de transit des conteneurs.

— Terminal roulier (Ro-Ro) Ce terminal est destiné au trafic des véhicules roulants, tels que les voitures, les camions, les remorques et les engins industriels. Il permet un embarquement et un débarquement rapides grâce à des infrastructures adaptées.

— Terminal minéralier Il prend en charge les marchandises solides non conteneurisées, notamment les céréales, le charbon, les minerais et d'autres matières premières destinées aux activités industrielles.

— Terminal polyvalent Il accueille différents types de marchandises qui ne relèvent pas des terminaux spécialisés, notamment les colis lourds, les marchandises diverses et certains équipements industriels.

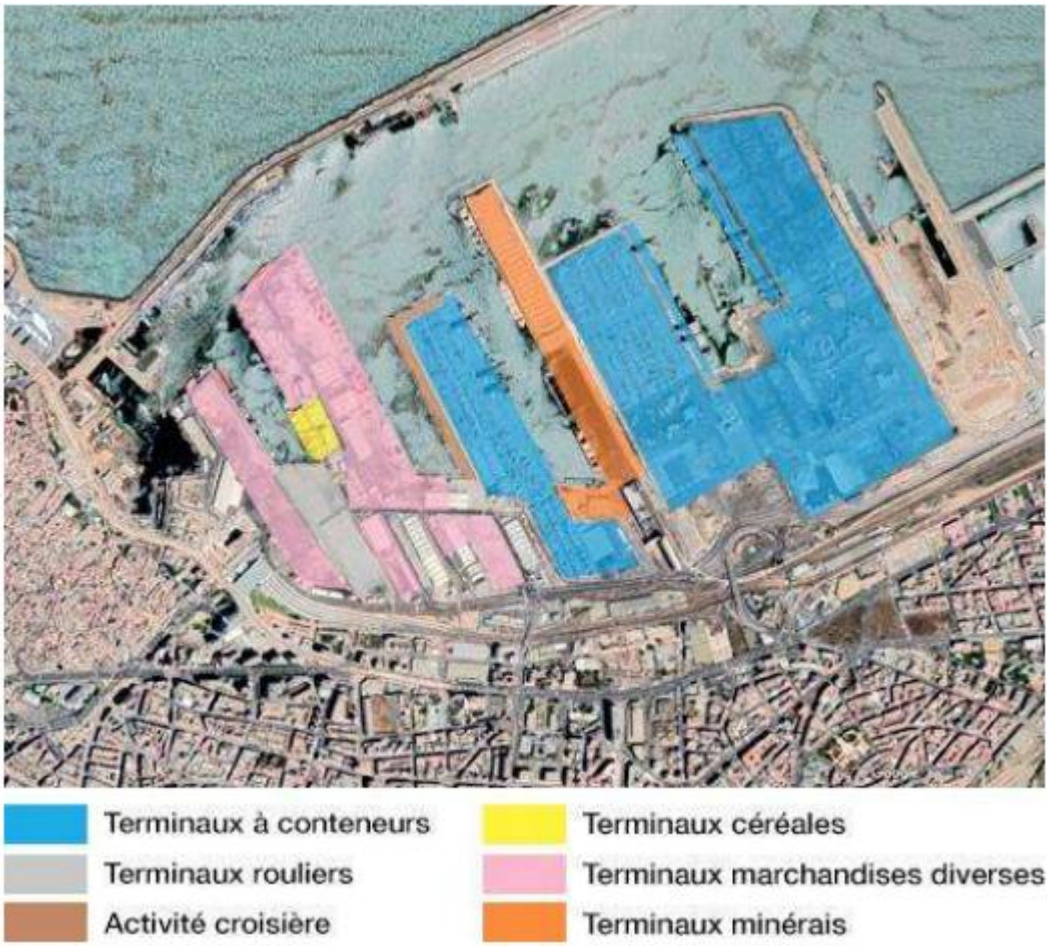


Figure 8 Plan des terminaux du Port de Casablanca.

L'organisation du Port de Casablanca repose ainsi sur une répartition fonctionnelle des terminaux selon la nature des trafics traitées. Cette spécialisation permet d'optimiser les opérations portuaires, d'améliorer la productivité des infrastructures et de répondre efficacement aux besoins des différents acteurs du transport maritime.

1.2.5 Division Système d'information

La Division Système d'Information (DSI) joue un rôle essentiel dans le fonctionnement de Marsa Maroc en assurant la gestion, le développement et l'évolution des systèmes d'information de l'entreprise. Elle accompagne les différents métiers en mettant à leur disposition des solutions informatiques permettant d'améliorer la performance des

activités portuaires, la circulation de l'information ainsi que la prise de décision. La DSI veille également au bon fonctionnement des infrastructures informatiques, à la disponibilité des applications métiers et à la sécurité des données. Elle participe activement à la transformation digitale de l'entreprise en intégrant des technologies innovantes destinées à optimiser les processus opérationnels et à répondre aux besoins croissants des utilisateurs.

Les principales missions de la Division Système d'Information peuvent être résumées comme suit :

— Développement et maintenance des applications métiers : conception, évolution et maintenance des solutions informatiques utilisées par les différents services de l'entreprise.

— Administration des systèmes et des infrastructures : gestion des serveurs, des réseaux et des équipements informatiques afin de garantir la disponibilité des services.

— Gestion des bases de données : administration, sécurisation et maintien de l'intégrité des données utilisées par les applications de l'entreprise.

— Sécurité des systèmes d'information : mise en œuvre de mesures visant à protéger les systèmes, les données et les infrastructures contre les risques informatiques.

— Accompagnement de la transformation digitale : intégration de nouvelles technologies et amélioration continue des outils numériques afin d'optimiser les processus métiers.

C'est au sein de cette division que s'est déroulé le stage présenté dans ce rapport. Réalisé dans un contexte de transformation digitale, ce stage porte sur le développement d'une solution basée sur l'intelligence artificielle visant à automatiser la détection et la reconnaissance des matricules tracteur , remorque des conteneurs à partir d'images capturer via un caméra connecté par réseau, contribuant ainsi à l'amélioration des opérations portuaires , ainsi que le développement d'une application mobile .



Chapitre 2

CONDUITE DU PROJET ET SPECIFICATION DES BESOINS





1.2 Cahier de Charge

1.2.1 Contexte du projet

Dans le cadre de ses activités industrielles et logistiques, Marsa Maroc gère quotidiennement des opérations impliquant des véhicules de transport, des chauffeurs, des marchandises et diverses prestations. Le processus de pesage constitue une étape clé dans le contrôle et la traçabilité des mouvements de marchandises au niveau des sites concernés.

Dans ce contexte, le projet **MIZAN – Gate de Pesage** a été mis en place afin de proposer une solution numérique permettant de moderniser et de sécuriser le processus de pesage. L'objectif est de centraliser les informations liées aux opérations, d'automatiser certaines étapes d'identification et de contrôle, et d'améliorer la traçabilité des opérations réalisées au niveau du poste de pesage.

La solution développée s'appuie sur une architecture composée d'une application web de supervision, d'une application mobile destinée aux opérations sur le terrain et de plusieurs services backend assurant la gestion des données et la communication entre les différents composants du système.

1.2.2 Problématique

Le processus de gestion des opérations de pesage implique plusieurs intervenants et étapes successives. La manipulation de ces informations peut devenir complexe lorsque certaines opérations sont réalisées manuellement ou lorsque les différentes données ne sont pas centralisées.

Plusieurs problématiques peuvent alors apparaître :

- la saisie manuelle des matricules des véhicules peut entraîner des erreurs ;
- l'identification du chauffeur nécessite des contrôles supplémentaires ;
- les informations relatives aux camions, chauffeurs, prestations et marchandises doivent être correctement associées ;
- le contrôle des autorisations d'accès à une prestation doit être assuré avant certaines opérations ;
- les opérations d'entrée et de sortie doivent être correctement enregistrées et historisées ;
- les données de pesée doivent être associées aux différents éléments de l'opération ;
- la génération et la consultation des tickets doivent être facilitées ;

- les différents acteurs doivent pouvoir disposer d'informations cohérentes et actualisées ;
- le suivi des opérations nécessite une meilleure visibilité pour les agents chargés de la supervision.

La problématique principale peut ainsi être formulée de la manière suivante :

Comment concevoir et mettre en œuvre une solution numérique permettant d'automatiser, de sécuriser et de centraliser le processus de pesage, tout en assurant l'identification des véhicules et des chauffeurs, le contrôle des prestations et des autorisations, ainsi que la traçabilité des opérations ?

Le projet **MIZAN** répond à cette problématique en proposant une plateforme permettant de gérer les différentes étapes du processus de pesage à travers une interface centralisée, des services backend et des mécanismes d'identification et de communication en temps réel.

1.2.3 Vue d'ensemble du projet

La solution repose sur plusieurs composants complémentaires :

- une **application web de supervision** destinée aux agents et responsables du poste de pesage ;
- une **application mobile PDA** destinée aux opérations réalisées sur le terrain ;
- un **backend centralisé** assurant la gestion de la logique métier et des échanges entre les différentes interfaces ;
- une **base de données** permettant de centraliser les informations relatives aux véhicules, chauffeurs, prestations, marchandises et pesées ;
- un **service OCR** permettant l'identification automatique des plaques d'immatriculation ;
- un système basé sur les **codes QR** pour faciliter l'identification et le suivi des opérations.

La solution a ainsi été pensée comme un système centralisé dans lequel les différentes opérations sont enregistrées et accessibles selon les besoins des utilisateurs.

1.2.4 Objectif général

L'objectif général de MIZAN est de mettre en place un processus de pesage plus automatisé, structuré et traçable.

Le système doit notamment permettre de :

- identifier automatiquement ou manuellement les véhicules ;

- gérer les informations relatives aux tracteurs et remorques ;
- identifier et vérifier les chauffeurs ;
- gérer les prestations associées aux véhicules ;
- contrôler les autorisations d'accès aux prestations ;
- enregistrer les pesées d'entrée et de sortie ;
- générer des tickets associés aux opérations ;
- utiliser des codes QR pour faciliter le suivi des dossiers ;
- centraliser l'historique des opérations ;
- assurer la communication en temps réel entre les différents composants du système ;
- fournir aux responsables une vue globale de l'activité du poste de pesage.

1.2.5 Principe général de fonctionnement

Le fonctionnement de MIZAN repose sur une succession d'étapes permettant de contrôler chaque opération de pesage.

Lorsqu'un véhicule se présente au poste de pesage, son identification peut être réalisée à partir de sa plaque d'immatriculation. Le système utilise un service OCR pour détecter et lire les plaques du tracteur et, lorsque cela est nécessaire, de la remorque.

Les informations récupérées sont ensuite comparées aux données enregistrées dans la base de données. Le système peut ainsi retrouver les informations relatives au véhicule et aux éléments associés.

Une étape de vérification du chauffeur peut également être réalisée à l'aide de son identité et de son code QR. Les informations du chauffeur peuvent alors être consultées avant la poursuite de l'opération.

Le système prend également en compte la prestation concernée ainsi que les autorisations associées au véhicule. Cette vérification permet de contrôler que le véhicule est autorisé à effectuer l'opération correspondante.

Une fois les différentes informations vérifiées, le poids du véhicule est récupéré. Le système attend la stabilisation de la mesure avant d'enregistrer la pesée.

L'opération est ensuite enregistrée dans la base de données et un ticket associé à la pesée peut être généré avec un code QR permettant de retrouver les informations de l'opération.

Les différentes informations sont ensuite disponibles au niveau de l'interface de supervision afin de permettre le suivi et la traçabilité des opérations.

1.2.6 Modules fonctionnels de la solution

Le système MIZAN est organisé autour de plusieurs modules fonctionnels.

Module	Fonction principale
Gestion des camions	Gestion des véhicules, tracteurs, remorques et matricules
Gestion des chauffeurs	Enregistrement et consultation des informations des chauffeurs
Identification OCR	Détection et lecture automatique des plaques d'immatriculation
Gestion des prestations	Gestion des prestations associées aux opérations
Gestion des autorisations	Vérification des autorisations des véhicules pour une prestation
Gestion de la pesée	Enregistrement des pesées d'entrée et de sortie
Gestion des tickets	Génération et consultation des tickets associés aux pesées
Gestion des codes QR	Identification et accès rapide aux informations liées aux opérations
Application PDA	Réalisation et suivi des opérations depuis un terminal mobile
Supervision	Suivi global des opérations et consultation de l'historique
Gestion des marchandises	Association des marchandises aux opérations de pesage
Gestion des rendez-vous	Gestion des rendez-vous liés aux opérations et aux chauffeurs

1.2.6.1 Besoins fonctionnels

Les besoins fonctionnels décrivent les principales fonctionnalités que la solution MIZAN doit fournir afin de répondre aux exigences du processus de pesage.

Gestion des véhicules

Le système doit permettre l'enregistrement des véhicules ainsi que la gestion de leurs informations associées. Il doit prendre en charge la distinction entre les tracteurs et les remorques, tout en permettant d'associer un matricule à chaque véhicule. La recherche d'un véhicule à partir de sa matricule doit être possible, et l'historique des opérations liées à un véhicule donné doit pouvoir être consulté à tout moment.

Identification automatique des plaques

Le système doit intégrer un mécanisme d'identification automatique des plaques d'immatriculation. Cette fonctionnalité repose sur un service OCR utilisant un modèle de détection et de lecture des plaques. Elle doit permettre de capturer l'image de la plaque, de détecter celle du tracteur et, lorsque cela est nécessaire, celle de la remorque. Le système doit ensuite extraire le numéro d'immatriculation, normaliser les données récupérées et rechercher le véhicule correspondant dans la base de données.

Gestion des chauffeurs

Le système doit permettre de gérer les informations relatives aux chauffeurs intervenant dans les opérations de pesage. Il doit notamment offrir la possibilité d'enregistrer un nouveau chauffeur, de consulter ses informations et de l'identifier à l'aide d'un code QR. Avant certaines opérations, l'identité du chauffeur doit pouvoir être vérifiée.

Gestion des prestations

La solution doit permettre de gérer les prestations liées aux opérations de pesage. Elle doit offrir la possibilité de consulter les prestations, d'associer une prestation à un véhicule et de vérifier l'état d'une prestation à tout moment. Le système doit également contrôler les autorisations associées à chaque prestation et permettre de retrouver une prestation à partir des informations disponibles dans le système.

Gestion des autorisations

Le système doit contrôler l'autorisation d'un véhicule à effectuer une prestation donnée. Une autorisation doit pouvoir être définie pour un véhicule et associée à une prestation spécifique. Lorsqu'une opération est effectuée, le système vérifie que cette autorisation est active avant de permettre la poursuite du processus, lorsque cette vérification est requise.

Gestion des opérations de pesée

Le système doit permettre d'enregistrer les différentes étapes d'une opération de pesage. Il doit notamment offrir la possibilité d'enregistrer une pesée d'entrée, de récupérer le poids du véhicule et de vérifier la stabilité de la mesure. La pesée doit être associée au véhicule concerné et, lorsque les informations sont disponibles, au chauffeur, à la prestation et à la marchandise. Une pesée de sortie doit également pouvoir être

enregistrée, et toutes les informations relatives à l'opération doivent être conservées dans l'historique.

Gestion des tickets et codes QR

Après l'enregistrement d'une opération, le système doit permettre de générer un ticket associé à la pesée. Ce ticket doit contenir les informations nécessaires au suivi de l'opération et intégrer un code QR permettant de retrouver rapidement les données correspondantes lors d'une consultation ultérieure.

Supervision et suivi

L'application web doit fournir une interface de supervision permettant aux utilisateurs autorisés de consulter les opérations de pesage, de suivre les entrées et sorties des véhicules et de consulter les prestations associées. Elle doit également permettre de suivre l'état des opérations en cours, de consulter l'historique complet et de visualiser les informations mises à jour en temps réel.

Communication en temps réel

Le système doit permettre la transmission en temps réel de certaines informations entre le backend, l'application web et le PDA. Cette communication permet notamment de mettre à jour automatiquement les informations relatives aux pesées et aux changements d'état, sans nécessiter une intervention manuelle permanente de l'utilisateur.

1.2.6.2 Besoins non fonctionnels

Les besoins non fonctionnels définissent les contraintes de qualité auxquelles la solution MIZAN doit répondre. Ils concernent notamment la sécurité, les performances, la fiabilité, l'ergonomie et la capacité d'évolution du système.

Sécurité

La solution doit assurer la protection des données et limiter l'accès aux fonctionnalités sensibles. Les principaux objectifs consistent à sécuriser l'authentification des utilisateurs, à protéger les informations enregistrées dans la base de données et à contrôler les accès aux différentes fonctionnalités. Le système doit également éviter les manipulations non autorisées et assurer la confidentialité des informations relatives aux chauffeurs, aux véhicules et aux opérations de pesage.

Disponibilité

Le système doit rester accessible pendant les périodes d'utilisation du poste de pesage. Les différents composants doivent pouvoir communiquer correctement afin de permettre la continuité des opérations de supervision et de pesage, même en cas de sollicitations importantes ou de variations de charge.

Ergonomie

Les interfaces doivent être simples et compréhensibles pour faciliter le travail des agents. L'application doit notamment proposer une organisation claire des informations, des étapes facilement identifiables et des messages indiquant les erreurs ou les validations. L'interface doit également être adaptée à l'utilisation sur ordinateur et sur terminal mobile, afin d'assurer une expérience utilisateur cohérente quel que soit le support utilisé.

Traçabilité

Chaque opération importante doit pouvoir être retrouvée dans le système. Les informations relatives aux pesées, aux véhicules, aux chauffeurs, aux prestations et aux tickets doivent être conservées afin de permettre la consultation de l'historique et le suivi détaillé des opérations. Cette traçabilité constitue un élément essentiel pour le contrôle, l'audit et l'analyse des activités du poste de pesage.

Maintenabilité

L'architecture doit être organisée de manière modulaire afin de faciliter la maintenance et les évolutions futures. La séparation entre l'interface utilisateur, le backend, la base de données et les services externes permet de modifier ou d'améliorer un composant sans remettre en cause l'ensemble de la solution. Cette approche favorise également la lisibilité du code et la gestion des corrections ou des améliorations.

Compatibilité

Les différents composants doivent pouvoir communiquer à travers des interfaces et des protocoles adaptés. L'utilisation d'API REST et de communications en temps réel permet notamment de connecter l'application web, le PDA et les services backend de manière homogène. Cette compatibilité assure l'interopérabilité entre les différents modules et facilite l'intégration de futurs composants ou services externes.

1.2.7 Technologies de développement

- **React** : développement de l'interface web de supervision.
- **React Native et Expo** : développement de l'application PDA.
- **JavaScript** : langage principal utilisé pour les interfaces et une partie de la logique applicative.
- **Node.js** : environnement d'exécution du backend.
- **Express.js** : développement des API REST.
- **API REST** : Communication backend ↔ Web/PDA

- **Sequelize** : gestion des modèles et communication avec la base de données.
- **PostgreSQL** : stockage et gestion des données du système.
- **Socket.IO** : communication en temps réel entre le serveur et les interfaces clientes.

1.2.8 Technologies liées à l'intelligence artificielle et à l'OCR

Le projet intègre un service d'identification automatique des plaques d'immatriculation basé sur la vision par ordinateur et l'OCR.

Roboflow est utilisé pour la préparation et l'exploitation des workflows de détection et de lecture des plaques.

En complément, **Google Colab** peut être utilisé comme environnement d'expérimentation pour les traitements liés à l'intelligence artificielle. Il permet notamment de :

- préparer et analyser les jeux de données ;
- expérimenter différents traitements d'images ;
- tester des modèles de détection ;
- évaluer les résultats obtenus ;
- comparer différentes configurations ;
- effectuer des expérimentations Python dans un environnement cloud.

Cette utilisation permet de séparer la phase d'expérimentation et d'analyse de la phase d'intégration dans l'application MIZAN.

1.2.9 Architecture fonctionnelle de la solution

MIZAN repose sur une architecture composée de plusieurs couches complémentaires.

➢ **Couche présentation**

Cette couche regroupe les interfaces utilisées par les différents utilisateurs.

L'**application web** permet notamment aux agents et responsables de consulter les informations, suivre les pesées et superviser l'activité du poste.

L'**application PDA** permet quant à elle de réaliser certaines opérations directement depuis un terminal mobile et de communiquer avec le système central.

➢ **Couche applicative**

Cette couche est assurée par le backend du projet. Elle centralise la logique métier et assure les échanges entre les interfaces, la base de données et les services externes.

Le backend expose des API permettant notamment de gérer :

- les camions ; les chauffeurs ;
- les prestations ; les autorisations ;
- les marchandises ; les pesées ;
- les tickets ; les utilisateurs.

Une communication temps réel est également mise en place afin de transmettre certaines mises à jour entre le backend, l'interface web et les applications clientes.

➢ **Couche données et services**

Cette couche regroupe la base de données ainsi que les services complémentaires utilisés par l'application.

La base de données permet de conserver les informations nécessaires au fonctionnement du système et d'assurer la traçabilité des opérations.

Le service OCR constitue un composant complémentaire chargé de l'identification des plaques d'immatriculation à partir des images capturées.

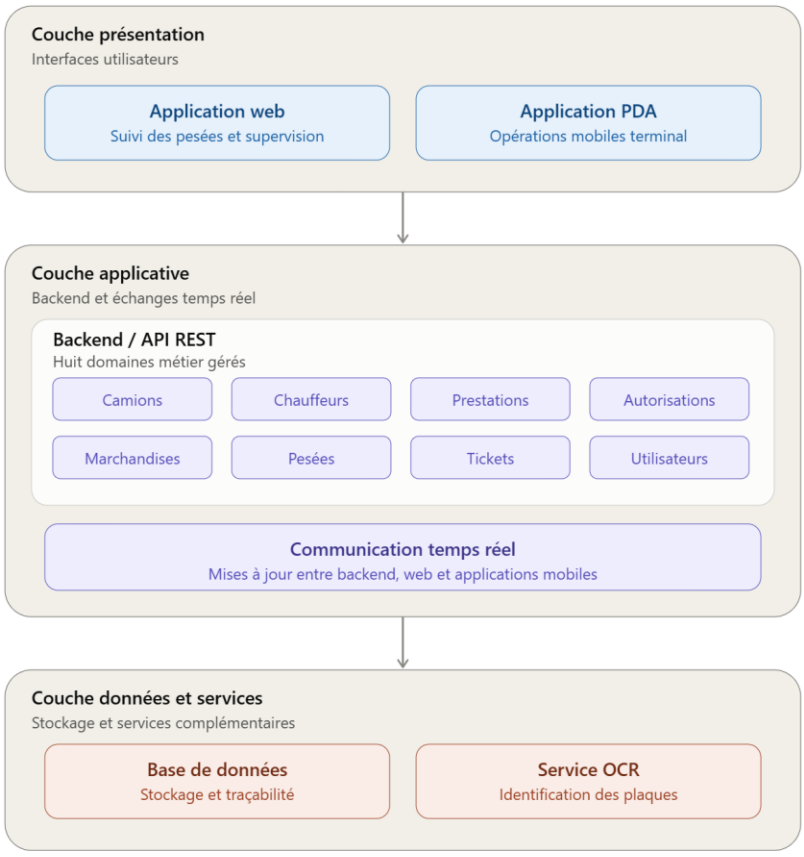


Figure 9 Architecture fonctionnelle de la solution

Conclusion

L'étude des besoins a permis de définir les principales fonctionnalités et contraintes de la solution MIZAN – Gate de Pesage.

Le cahier des charges met en évidence la nécessité d'une solution capable de centraliser les informations relatives aux véhicules, chauffeurs, prestations, autorisations, marchandises et opérations de pesage. L'automatisation de l'identification des plaques, l'utilisation des codes QR, la gestion des autorisations et la supervision en temps réel constituent les principaux éléments fonctionnels de la solution.

Les besoins non fonctionnels définissent quant à eux les exigences de sécurité, de fiabilité, de performance, d'ergonomie, de traçabilité, de maintenabilité et d'évolutivité nécessaires au bon fonctionnement du système.

Cette analyse constitue la base de la phase de conception. Elle permet de définir l'architecture technique, les différents composants logiciels, le modèle de données ainsi que les interactions entre les acteurs et les différents modules de MIZAN.

La prochaine étape consiste donc à concevoir la solution à partir des besoins identifiés, notamment à travers les diagrammes UML, l'architecture générale du système et la conception de la base de données.



Chapitre 3

CONCEPTION ET ARCHITECTURE TECHNIQUE





3.1 Modélisation UML

Pour concevoir le système MIZAN de manière rigoureuse, nous avons eu recours à la modélisation UML (Unified Modeling Language). Cette approche nous a permis de visualiser l'ensemble du système avant de passer au développement, en représentant de façon claire les fonctionnalités attendues, les différents acteurs impliqués et leurs interactions.

L'utilisation des diagrammes UML répond à plusieurs objectifs dans ce projet. D'une part, elle facilite la compréhension du processus de pesage dans sa globalité. D'autre part, elle aide à définir précisément le rôle de chaque acteur et à structurer les composants logiciels. Enfin, elle garantit l'alignement entre les besoins exprimés, l'architecture technique retenue, le modèle de données et les interfaces des applications web et mobile.

Afin de couvrir tous les aspects du système, six types de diagrammes ont été produits :

- Le **diagramme de cas d'utilisation** recense les acteurs et leurs interactions avec les fonctionnalités du système ;
- Le **diagramme de classes** modélise l'ossature statique du système en définissant les classes, leurs propriétés et leurs associations ;
- Le **diagramme de séquence** décrit les interactions dynamiques entre les composants durant l'exécution d'un scénario, tel que le processus de pesée ;
- Le **diagramme d'activité** retrace le cheminement des étapes successives du processus de pesage ;
- Le **diagramme d'état-transition** expose les différents états traversés par une opération de pesée tout au long de son cycle de vie ;
- Le **diagramme de déploiement** schématise l'architecture physique en montrant la répartition des éléments logiciels et matériels.

Ces diagrammes forment un ensemble cohérent qui offre une vision d'ensemble du système MIZAN. Ils servent de fil conducteur tout au long des phases de développement, de validation et de mise en production.

3.1.1 Diagramme de cas d’utilisation

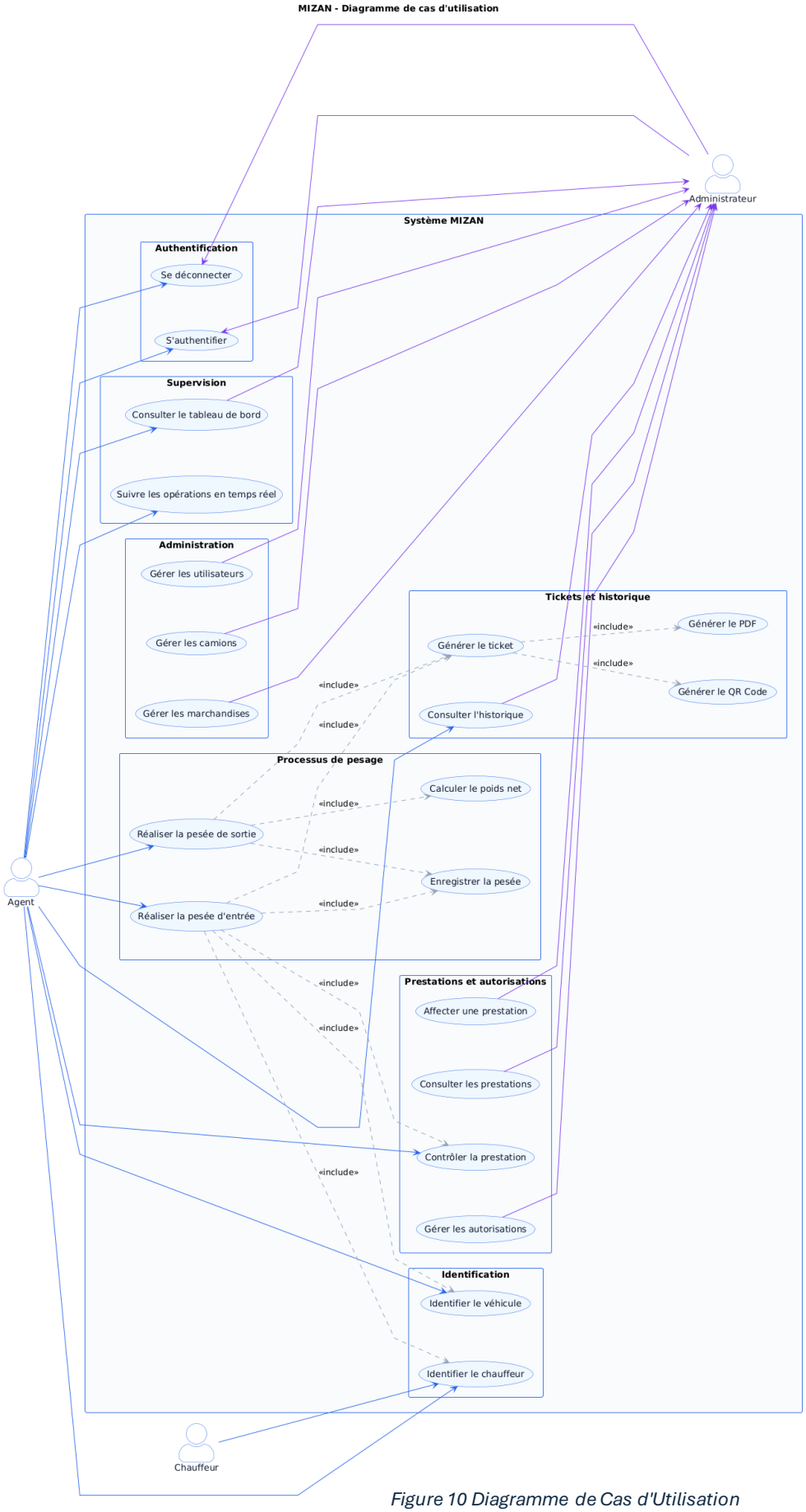


Figure 10 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation présenté ci-dessous modélise les interactions entre les acteurs du système MIZAN et les fonctionnalités offertes par la plateforme. Il permet d'identifier clairement qui fait quoi dans le système et de comprendre l'organisation générale des fonctionnalités.

Trois acteurs principaux interagissent avec le système MIZAN. Le premier est l'Agent, qui correspond à l'opérateur responsable des opérations de pesage sur le terrain. Il réalise les pesées d'entrée et de sortie, identifie les véhicules et les chauffeurs, et consulte les informations de supervision. Le deuxième acteur est le Chauffeur, qui intervient uniquement pour s'identifier lors du processus de pesage, généralement via un code QR ou un badge personnel. Le troisième acteur est l'Administrateur, qui dispose d'un accès complet aux fonctionnalités de gestion et de configuration du système. Il gère les utilisateurs, les camions, les marchandises, les prestations et les autorisations.

Les cas d'utilisation sont organisés en plusieurs packages fonctionnels au sein du système MIZAN. Le package Authentification regroupe les fonctionnalités de connexion et de déconnexion des utilisateurs. Tous les acteurs doivent s'authentifier avant d'accéder aux fonctionnalités du système. Le package Identification comprend l'identification du véhicule par lecture automatique de la plaque et l'identification du chauffeur par code QR ou badge. Ces deux cas d'utilisation sont inclus dans le processus de pesée d'entrée.

Le package Processus de pesage constitue le cœur fonctionnel du système. Il comprend la pesée d'entrée, la pesée de sortie, le calcul du poids net et l'enregistrement des données de pesée. La pesée d'entrée inclut l'identification du véhicule, l'identification du chauffeur, le contrôle de la prestation et l'enregistrement des données. La pesée de sortie inclut le calcul du poids net, l'enregistrement et la génération du ticket. Le package Tickets et historique regroupe les fonctionnalités liées à la génération des tickets, incluant la création du QR Code et du PDF, ainsi que la consultation de l'historique des opérations.

Le package Supervision offre aux agents et administrateurs la possibilité de consulter le tableau de bord et de suivre les opérations en temps réel. Le package Prestations et autorisations permet de consulter les prestations, d'affecter une prestation à un véhicule, de contrôler la prestation avant une opération et de gérer les autorisations d'accès. Enfin, le package Administration est réservé à l'administrateur et lui permet de gérer les utilisateurs, les camions et les marchandises.

Les relations de type include, représentées par des flèches pointillées, indiquent qu'un cas d'utilisation en inclut obligatoirement un autre. Par exemple, la pesée d'entrée inclut l'identification du véhicule, l'identification du chauffeur, le contrôle de la prestation, l'enregistrement de la pesée et la génération du ticket. De même, la génération du ticket inclut la création du QR Code et du PDF.

3.1.3 Diagramme de Classe

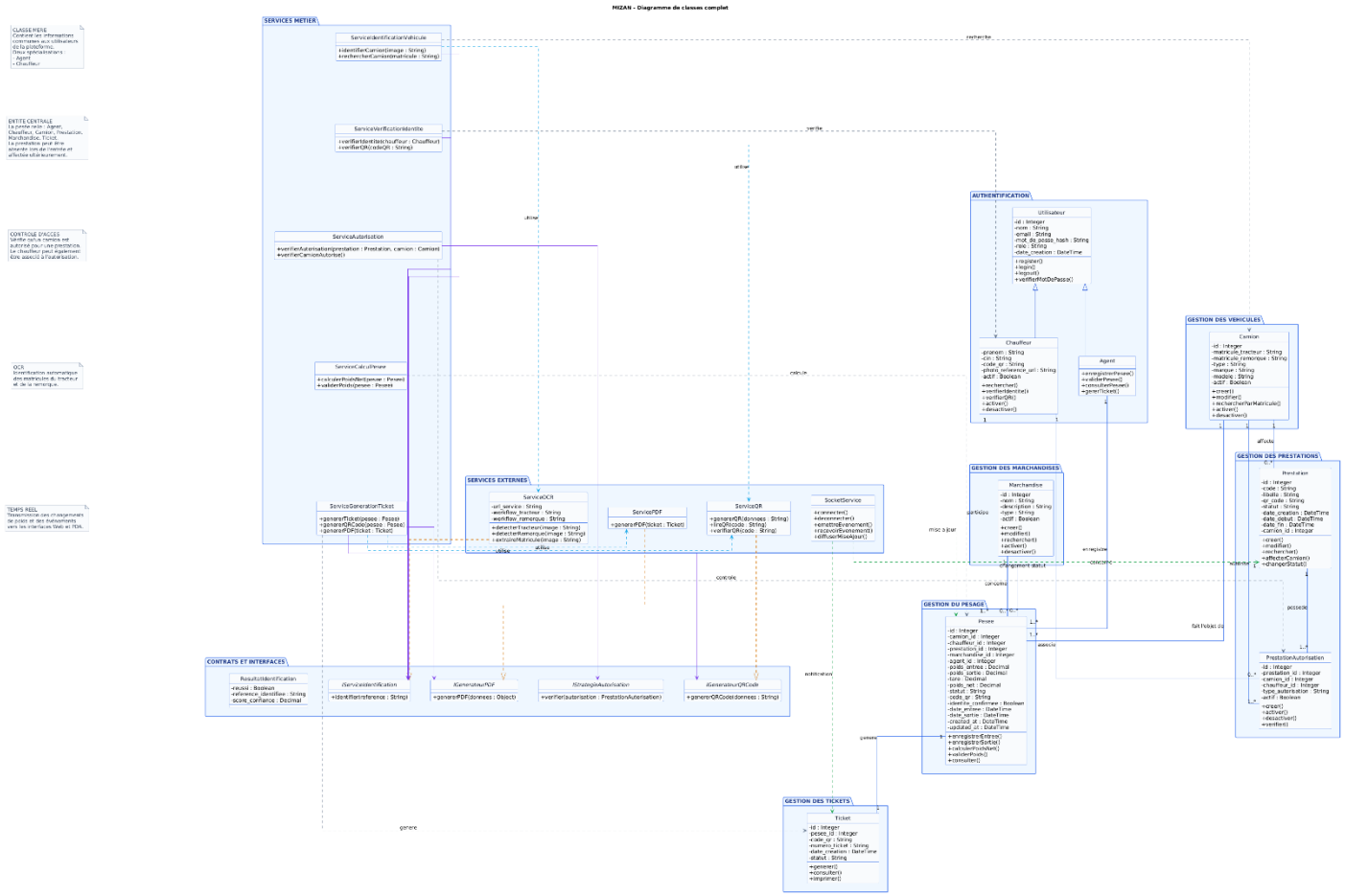


Figure 11 Diagramme de Classes

Le diagramme est organisé en quatre bandes horizontales structurées. La première bande regroupe les classes métier représentant les entités fondamentales du domaine. La deuxième bande contient les services métier qui encapsulent la logique fonctionnelle. La troisième bande présente les contrats et interfaces qui définissent les abstractions du système. La quatrième et dernière bande rassemble les services externes qui assurent l'intégration avec des composants techniques spécialisés.

La bande métier commence par le package Authentification, qui contient la classe Utilisateur servant de classe mère aux classes Agent et Chauffeur. La classe Utilisateur centralise les informations communes telles que l'identifiant, le nom, l'email, le mot de passe haché et le rôle. Elle définit également les méthodes d'authentification comme l'inscription, la connexion et la déconnexion. La classe Agent spécialise Utilisateur et ajoute des méthodes spécifiques à la gestion des pesées, notamment l'enregistrement, la validation et la consultation des pesées ainsi que la gestion des tickets. La classe Chauffeur contient des attributs spécifiques comme le prénom, le numéro de carte

d'identité, le code QR personnel et la photo de référence. Elle offre des méthodes pour la recherche, la vérification d'identité et la gestion de l'état actif ou inactif d'un chauffeur.

Le package Gestion des véhicules contient la classe Camion, qui représente les véhicules de transport. Elle stocke les matricules du tracteur et de la remorque, ainsi que des informations complémentaires comme le type, la marque et le modèle. Elle propose des méthodes pour la création, la modification, la recherche par matricule et la gestion de l'état actif. Le package Gestion des prestations comprend deux classes. La classe Prestation représente une opération de transport associée à un camion et contient des informations comme le code, le libellé, le code QR, le statut et les dates de début et de fin. La classe PrestationAutorisation modélise les autorisations d'accès et permet de lier une prestation à un camion et éventuellement à un chauffeur. Elle inclut une méthode de vérification pour contrôler si une autorisation est active.

Le package Gestion des marchandises contient la classe Marchandise, qui décrit les types de biens transportés avec un nom, une description et un type. Le package Gestion du pesage contient la classe Pesee, qui constitue l'entité centrale du système. Elle relie l'ensemble des autres entités métier en stockant les références au camion, au chauffeur, à la prestation, à la marchandise et à l'agent. Elle conserve également les poids d'entrée et de sortie, la tare, le poids net calculé, le statut de l'opération, le code QR généré et les dates d'entrée et de sortie. Les méthodes principales permettent d'enregistrer l'entrée et la sortie, de calculer le poids net, de valider les données et de consulter les informations d'une pesée. Le package Gestion des tickets contient la classe Ticket, qui est générée après chaque pesée et contient un code QR, un numéro unique et un statut. Elle offre des méthodes pour générer, consulter et imprimer le ticket.

Les relations entre les classes métier sont clairement définies. Un agent peut enregistrer plusieurs pesées, tandis qu'un chauffeur participe à plusieurs pesées. Un camion fait l'objet de plusieurs pesées et peut être affecté à plusieurs prestations. Une prestation concerne zéro ou plusieurs pesées, car elle peut être absente lors de l'entrée et affectée ultérieurement. Une marchandise concerne également zéro ou plusieurs pesées. Chaque pesée génère exactement un ticket. Une prestation possède une ou plusieurs autorisations, et un camion peut être autorisé sur plusieurs prestations. Un chauffeur peut être associé à plusieurs autorisations.

La bande des services métier contient cinq classes qui encapsulent la logique fonctionnelle du système. Le **ServiceIdentificationVehicule** permet d'identifier un camion à partir d'une image et de rechercher un camion par son matricule. Le **ServiceVerificationIdentite** vérifie l'identité d'un chauffeur et valide son code QR. Le **ServiceAutorisation** contrôle si un camion est autorisé pour une prestation donnée. Le **ServiceCalculPesee** calcule le poids net à partir des données d'entrée et de sortie et valide la cohérence des poids. Le **ServiceGenerationTicket** génère un ticket à partir d'une pesée, produit le code QR associé et crée le document PDF imprimable.

La bande des contrats et interfaces définit les abstractions du système. L'interface **IServiceIdentification** spécifie la méthode d'identification d'une référence. L'interface **IGenerateurQRCode** définit la méthode de génération d'un code QR à partir de données. L'interface **IGenerateurPDF** spécifie la méthode de génération d'un document PDF. L'interface **IStrategieAutorisation** définit la méthode de vérification d'une autorisation. La classe ResultatIdentification modélise le résultat d'une opération d'identification avec un indicateur de succès, la référence identifiée et un score de confiance.

La bande des services externes regroupe les composants techniques spécialisés. Le **ServiceOCR** assure la détection et l'extraction des matricules à partir d'images capturées. Il propose des méthodes pour détecter le tracteur, détecter la remorque et extraire le matricule. Le **ServiceQR** gère la génération, la lecture et la vérification des codes QR. Le **ServicePDF** se charge de la génération des documents PDF à partir des données d'un ticket. Le **SocketService** assure la communication en temps réel entre le backend, l'application web et les terminaux mobiles. Il permet de se connecter, de se déconnecter, d'émettre et de recevoir des événements et de diffuser les mises à jour.

Les relations entre les services métier et les autres composants suivent des principes d'architecture propres. Les services métier dépendent des interfaces définies dans la bande des contrats, ce qui permet d'appliquer le principe d'inversion de dépendance. Par exemple, le **ServiceIdentificationVehicule** dépend de l'interface **IServiceIdentification**, et le **ServiceGenerationTicket** dépend des interfaces **IGenerateurQRCode** et IGenerateurPDF. Les services métier utilisent également les services externes correspondants. Le **ServiceIdentificationVehicule** utilise le ServiceOCR, le **ServiceVerificationIdentite** utilise le **ServiceQR**, et le **ServiceGenerationTicket** utilise à la fois le **ServiceQR** et le **ServicePDF**.

Les services métier agissent directement sur les classes métier. Le **ServiceIdentificationVehicule** recherche des camions, le **ServiceVerificationIdentite** vérifie les chauffeurs, le **ServiceAutorisation** contrôle les autorisations, le **ServiceCalculPesee** calcule les données de pesée et le **ServiceGenerationTicket** génère les tickets. Les services externes implémentent les interfaces correspondantes. Le ServiceOCR implémente **IServiceIdentification**, le **ServiceQR** implémente **IGenerateurQRCode** et le **ServicePDF** implémente **IGenerateurPDF**.

Le SocketService assure la communication en temps réel en diffusant les mises à jour des pesées, les changements de statut des prestations et les notifications liées aux tickets vers les interfaces web et mobiles. Cette fonctionnalité permet aux utilisateurs de visualiser les données actualisées sans intervention manuelle.

Ce diagramme de classes offre une vision complète et structurée de l'architecture logicielle du système MIZAN. Il sépare clairement les responsabilités entre les entités métier, les services fonctionnels, les abstractions et les services techniques. Cette

organisation facilite la maintenance, l'évolution et les tests du système tout en garantissant une conception robuste et évolutive.

3.1.4 Diagramme de Séquence

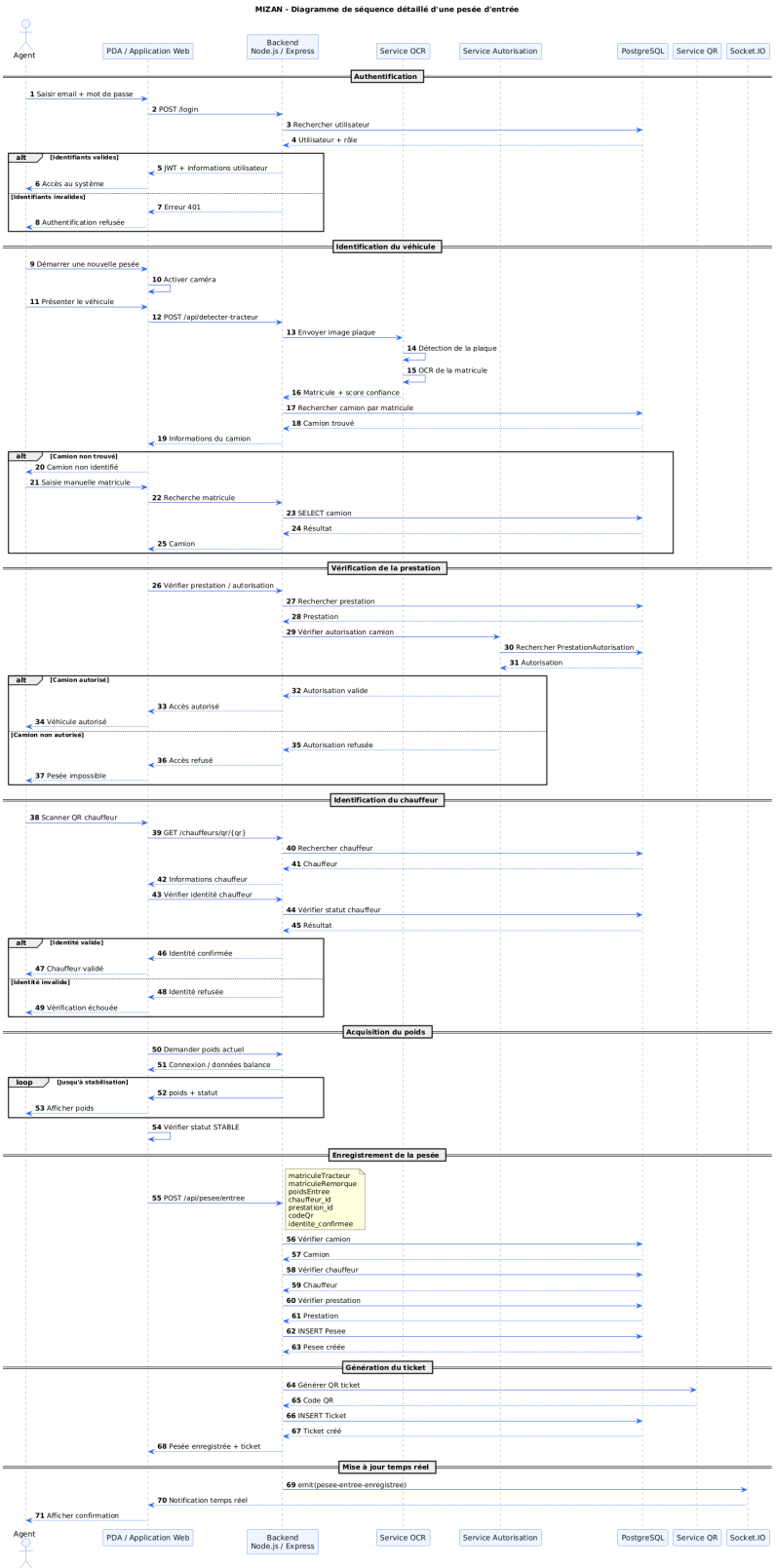


Figure 12 Diagramme de Séquence

Le diagramme met en scène sept participants principaux. L'Agent représente l'opérateur humain qui réalise la pesée sur le terrain. Le PDA ou Application Web constitue l'interface utilisateur utilisée par l'agent pour interagir avec le système. Le Backend Node.js Express assure la logique métier et l'orchestration des différents services. Le Service OCR est responsable de la détection et de la lecture des plaques d'immatriculation. Le Service Autorisation contrôle les autorisations d'accès des véhicules aux prestations. La base de données PostgreSQL stocke l'ensemble des données persistantes. Le Service QR génère les codes QR associés aux tickets. Enfin, le service Socket.IO assure la communication en temps réel entre les différents composants.

La séquence commence par la phase d'authentification. L'agent saisit son email et son mot de passe dans l'application, qui transmet ces informations au backend via une requête POST sur le point de terminaison login. Le backend interroge la base de données pour rechercher l'utilisateur correspondant et récupérer son rôle. Si les identifiants sont valides, le backend génère un jeton JWT et renvoie les informations de l'utilisateur à l'application, qui accorde alors l'accès au système. Dans le cas contraire, le backend renvoie une erreur 401 et l'authentification est refusée.

Une fois authentifié, l'agent démarre une nouvelle pesée en activant la caméra via l'application. Il présente ensuite le véhicule devant la caméra pour permettre la capture de l'image de la plaque. L'application envoie une requête POST au backend sur le point de terminaison detecter-tracteur, qui transmet l'image au service OCR. Ce dernier effectue la détection de la plaque puis l'extraction du matricule par reconnaissance optique de caractères. Il renvoie le matricule détecté accompagné d'un score de confiance au backend. Celui-ci interroge la base de données pour rechercher le camion correspondant à ce matricule. Si le camion est trouvé, ses informations sont renvoyées à l'application. Dans le cas où le camion n'est pas identifié automatiquement, l'agent peut saisir manuellement le matricule, ce qui déclenche une nouvelle recherche en base de données.

La phase suivante concerne la vérification de la prestation et des autorisations. L'application demande au backend de vérifier si le véhicule dispose d'une prestation et d'une autorisation valide. Le backend recherche la prestation dans la base de données, puis interroge le service Autorisation pour contrôler si le camion est autorisé pour cette prestation. Le service Autorisation recherche l'autorisation correspondante dans la table PrestationAutorisation. Si le camion est autorisé, le service renvoie une autorisation valide et l'accès est accordé. Sinon, l'autorisation est refusée et la pesée ne peut pas se poursuivre.

L'identification du chauffeur constitue l'étape suivante. L'agent scanne le code QR du chauffeur à l'aide de l'application, qui envoie une requête GET au backend avec le code QR en paramètre. Le backend recherche le chauffeur correspondant dans la base de données et renvoie ses informations. L'application demande ensuite au backend de

vérifier l'identité du chauffeur en contrôlant son statut dans la base de données. Si l'identité est valide, le chauffeur est confirmé et la pesée peut continuer. Dans le cas contraire, la vérification échoue et l'agent en est informé.

L'acquisition du poids débute ensuite. L'application demande au backend le poids actuel du véhicule. Le backend établit la connexion avec la balance et récupère les données de pesée. Une boucle s'exécute jusqu'à la stabilisation du poids, pendant laquelle le backend transmet régulièrement le poids et son statut à l'application, qui l'affiche à l'agent. Une fois le statut STABLE atteint, l'application peut procéder à l'enregistrement de la pesée.

L'enregistrement de la pesée d'entrée se fait via une requête POST envoyée par l'application au backend sur le point de terminaison pesee/entree. Cette requête contient l'ensemble des données nécessaires, notamment les matricules du tracteur et de la remorque, le poids d'entrée, l'identifiant du chauffeur, l'identifiant de la prestation, le code QR et l'indicateur de confirmation de l'identité. Le backend vérifie successivement l'existence du camion, du chauffeur et de la prestation dans la base de données. Si toutes les vérifications sont positives, il insère un nouvel enregistrement dans la table Pesee.

La génération du ticket intervient après l'enregistrement de la pesée. Le backend demande au service QR de générer un code QR contenant les informations de la pesée. Une fois le code QR généré, le backend insère un nouvel enregistrement dans la table Ticket avec le code QR et les autres informations associées. Le backend renvoie ensuite à l'application la confirmation que la pesée est enregistrée et que le ticket est généré.

La dernière phase concerne la mise à jour en temps réel. Le backend émet un événement pesee-entree-enregistree via le service Socket.IO, qui diffuse la notification à tous les clients connectés. L'application reçoit cette notification et affiche la confirmation de la pesée à l'agent. Cette fonctionnalité permet aux superviseurs connectés au tableau de bord de visualiser immédiatement la nouvelle pesée sans actualisation manuelle.

3.1.5 Diagramme d’activité

Le processus débute par l'authentification de l'agent. Celui-ci doit s'identifier auprès du système en fournissant ses identifiants. Une décision est alors prise pour vérifier la validité de l'authentification. Si l'authentification est invalide, le système affiche une erreur d'authentification et le processus s'arrête. Si l'authentification est valide, l'agent accède au module de pesée et peut démarrer une nouvelle opération.

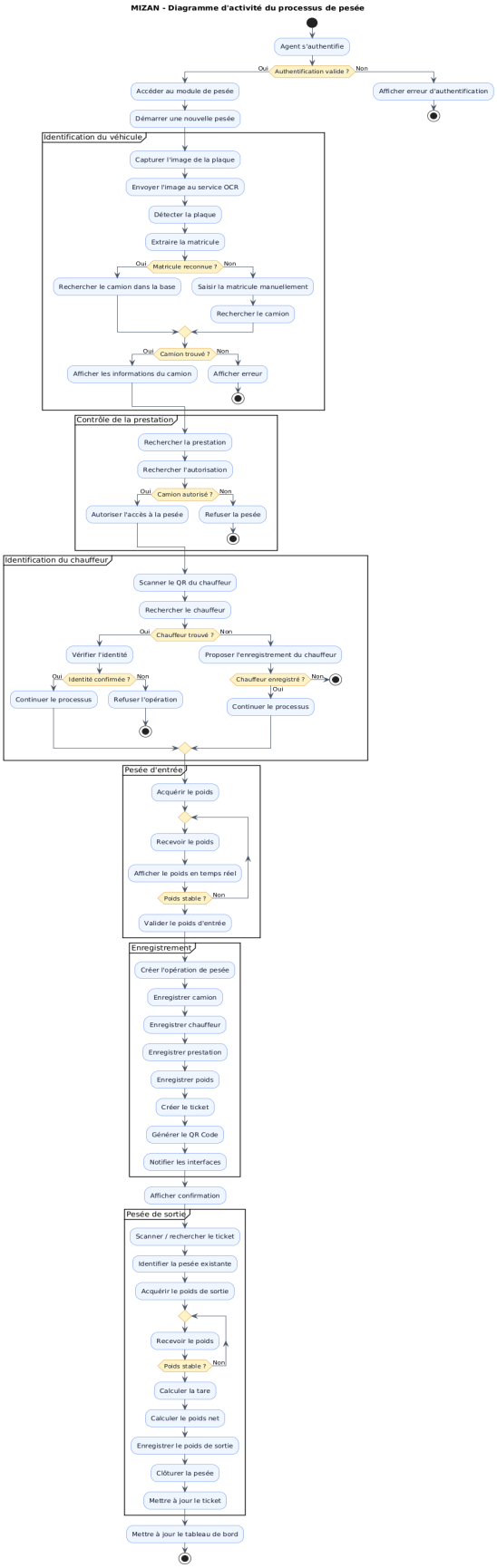


Figure 13 Diagramme d'activté

La première phase du processus est l'identification du véhicule. L'agent capture l'image de la plaque d'immatriculation du véhicule à l'aide de la caméra. Cette image est ensuite envoyée au service OCR pour traitement. Le service détecte la plaque puis extrait le matricule par reconnaissance optique de caractères. Une décision est prise pour vérifier si le matricule a été reconnu automatiquement. Si la reconnaissance est réussie, le système recherche directement le camion dans la base de données à partir du matricule extrait. Si la reconnaissance échoue, l'agent doit saisir manuellement le matricule, ce qui déclenche une recherche du camion dans la base de données. Une seconde décision vérifie si le camion a été trouvé. Si le camion n'est pas trouvé, le système affiche une erreur et le processus s'arrête. Si le camion est trouvé, ses informations sont affichées et le processus continue.

La phase suivante est le contrôle de la prestation. Le système recherche la prestation associée au véhicule puis recherche l'autorisation correspondante dans la base de données. Une décision vérifie si le camion est autorisé pour cette prestation. Si le camion n'est pas autorisé, la pesée est refusée et le processus s'arrête. Si le camion est autorisé, l'accès à la pesée est accordé et le processus se poursuit.

L'identification du chauffeur constitue la troisième phase. L'agent scanne le code QR du chauffeur à l'aide de l'application. Le système recherche ensuite le chauffeur dans la base de données à partir du code QR scanné. Une décision vérifie si le chauffeur a été trouvé. Si le chauffeur n'est pas trouvé, le système propose d'enregistrer un nouveau chauffeur. Si l'agent choisit de ne pas enregistrer le chauffeur, le processus s'arrête. Si le chauffeur est enregistré ou s'il a été trouvé dès le départ, le système procède à la vérification de son identité. Une décision vérifie si l'identité est confirmée. Si l'identité n'est pas confirmée, l'opération est refusée et le processus s'arrête. Si l'identité est confirmée, le processus continue.

La phase de pesée d'entrée débute ensuite. Le système acquiert le poids du véhicule en se connectant à la balance. Une boucle répétitive s'exécute tant que le poids n'est pas stable. À chaque itération, le système reçoit le poids actuel et l'affiche en temps réel sur l'interface. Une fois le poids stable, l'agent valide le poids d'entrée.

La phase d'enregistrement intervient après la validation du poids. Le système crée l'opération de pesée en enregistrant successivement les informations du camion, du chauffeur, de la prestation et du poids. Il crée ensuite le ticket associé à la pesée et génère le code QR correspondant. Le système notifie ensuite les différentes interfaces connectées de la nouvelle pesée enregistrée. Une confirmation est affichée à l'agent pour valider l'enregistrement de la pesée d'entrée.

La phase de pesée de sortie se déroule ultérieurement, lorsque le véhicule quitte le site. L'agent scanne ou recherche le ticket généré lors de la pesée d'entrée. Le système identifie la pesée existante associée à ce ticket. Il acquiert ensuite le poids de sortie du véhicule en se connectant à nouveau à la balance. Une boucle répétitive s'exécute tant

que le poids n'est pas stable, recevant et affichant le poids à chaque itération. Une fois le poids stable, le système calcule la tare en soustrayant le poids d'entrée du poids de sortie, puis calcule le poids net. Il enregistre le poids de sortie dans la base de données et clôture la pesée. Le ticket est mis à jour avec les informations complètes de la pesée.

Le processus se termine par la mise à jour du tableau de bord. Le système actualise les informations affichées sur le tableau de bord de supervision pour refléter la nouvelle pesée clôturée. Cette mise à jour permet aux superviseurs de visualiser l'état actualisé des opérations en temps réel.

3.1.6 Diagramme d'etat transition

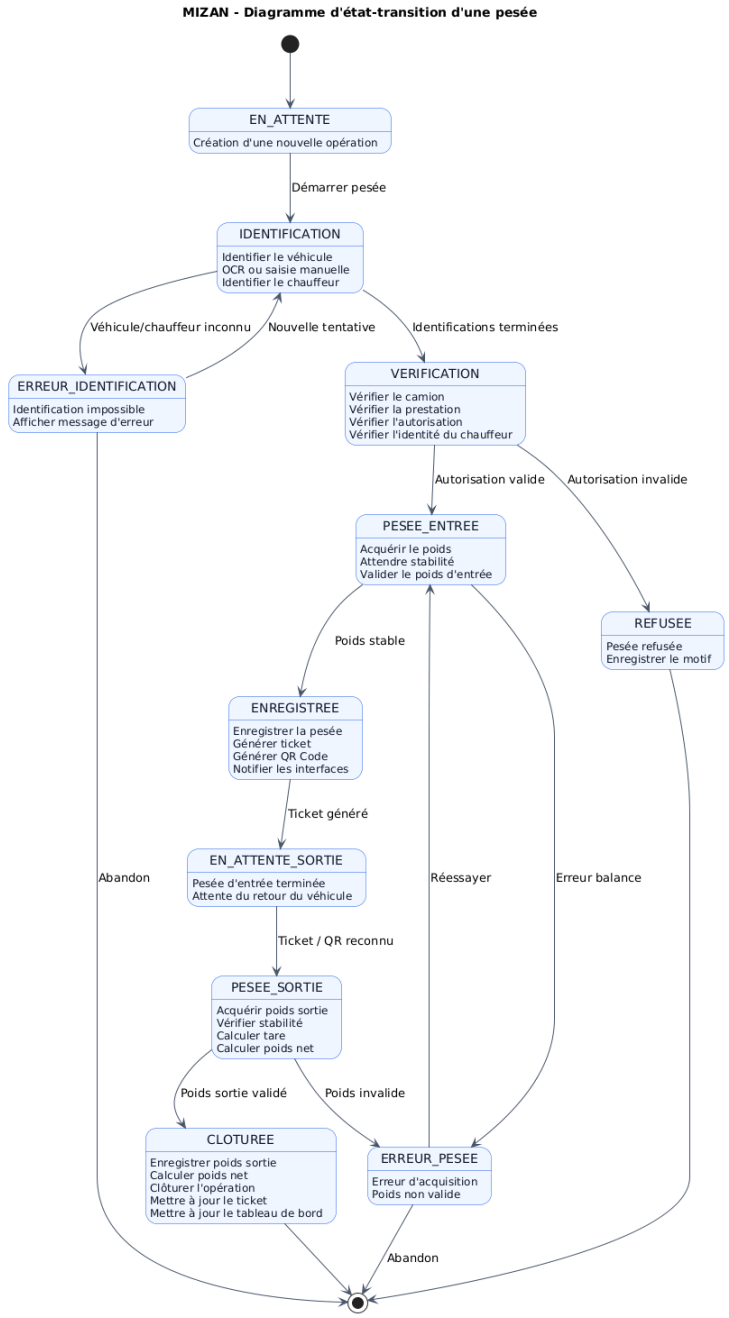


Figure 14 Diagramme d'état-transition

Le cycle de vie débute par l'état initial représenté par le point noir, qui conduit directement à l'état EN_ATTENTE. Dans cet état, une nouvelle opération de pesée est créée mais n'a pas encore démarré. Le système attend que l'agent initie le processus. La transition vers l'état suivant se produit lorsque l'agent démarre la pesée, ce qui fait passer l'opération à l'état IDENTIFICATION.

L'état IDENTIFICATION correspond à la phase d'identification des différents éléments de l'opération. Le système identifie le véhicule par OCR ou par saisie manuelle du matricule, puis identifie le chauffeur via son code QR. Deux transitions sont possibles depuis cet état. Si les identifications du véhicule et du chauffeur sont terminées avec succès, l'opération passe à l'état VERIFICATION. Si le véhicule ou le chauffeur est inconnu, l'opération passe à l'état ERREUR_IDENTIFICATION.

L'état ERREUR_IDENTIFICATION représente une situation d'échec de l'identification. Le système affiche un message d'erreur indiquant que l'identification est impossible. Deux issues sont alors possibles. L'agent peut choisir de faire une nouvelle tentative, ce qui fait retourner l'opération à l'état IDENTIFICATION. Ou bien l'agent peut abandonner l'opération, ce qui conduit à l'état final représenté par le cercle entouré.

L'état VERIFICATION correspond à la phase de contrôle des autorisations. Le système vérifie successivement le camion, la prestation associée, l'autorisation d'accès et l'identité du chauffeur. Deux transitions sont possibles depuis cet état. Si toutes les vérifications sont valides, notamment l'autorisation du camion pour la prestation, l'opération passe à l'état PESEE_ENTREE. Si l'autorisation est invalide, l'opération passe à l'état REFUSEE.

L'état REFUSEE représente le refus de la pesée en raison d'une autorisation invalide. Le système enregistre le motif du refus dans la base de données pour tracer l'incident. Cet état conduit directement à l'état final, car l'opération ne peut pas se poursuivre sans autorisation valide.

L'état PESEE_ENTREE correspond à la phase d'acquisition du poids d'entrée. Le système acquiert le poids en se connectant à la balance, attend la stabilisation de la mesure puis valide le poids d'entrée. Trois transitions sont possibles depuis cet état. Si le poids est stable et valide, l'opération passe à l'état ENREGISTREE. Si une erreur de balance survient ou si le poids n'est pas valide, l'opération passe à l'état ERREUR_PESEE.

L'état ERREUR_PESEE représente une erreur d'acquisition du poids. Le système indique que le poids n'est pas valide ou que l'acquisition a échoué. Deux issues sont possibles. Le système peut réessayer l'acquisition, ce qui fait retourner l'opération à l'état PESEE_ENTREE. Ou bien l'agent peut abandonner l'opération, ce qui conduit à l'état final.

L'état ENREGISTREE correspond à la phase d'enregistrement de la pesée d'entrée. Le système enregistre les données de la pesée dans la base de données, génère le ticket

associé, produit le code QR et notifie les interfaces connectées via Socket.IO. Une fois le ticket généré avec succès, l'opération passe à l'état EN_ATTENTE_SORTIE.

L'état EN_ATTENTE_SORTIE représente la période entre la pesée d'entrée et la pesée de sortie. La pesée d'entrée est terminée et le système attend le retour du véhicule pour la pesée de sortie. Cet état peut durer plusieurs heures selon le temps nécessaire au chargement ou au déchargement de la marchandise. La transition vers l'état suivant se produit lorsque l'agent scanne le ticket ou le code QR pour initier la pesée de sortie.

L'état PESEE_SORTIE correspond à la phase d'acquisition du poids de sortie. Le système acquiert le poids de sortie en se connectant à la balance, vérifie la stabilisation de la mesure, calcule la tare en soustrayant le poids d'entrée du poids de sortie, puis calcule le poids net. Deux transitions sont possibles depuis cet état. Si le poids de sortie est validé avec succès, l'opération passe à l'état CLOTUREE. Si le poids est invalide ou si une erreur survient, l'opération passe à l'état ERREUR_PESEE.

L'état CLOTUREE représente la phase finale de l'opération. Le système enregistre le poids de sortie dans la base de données, calcule et enregistre le poids net, clôture officiellement l'opération, met à jour le ticket avec les informations complètes et actualise le tableau de bord de supervision. Une fois toutes ces actions terminées, l'opération passe à l'état final, marquant la fin de son cycle de vie.

3.1.7 Diagramme de deploiement

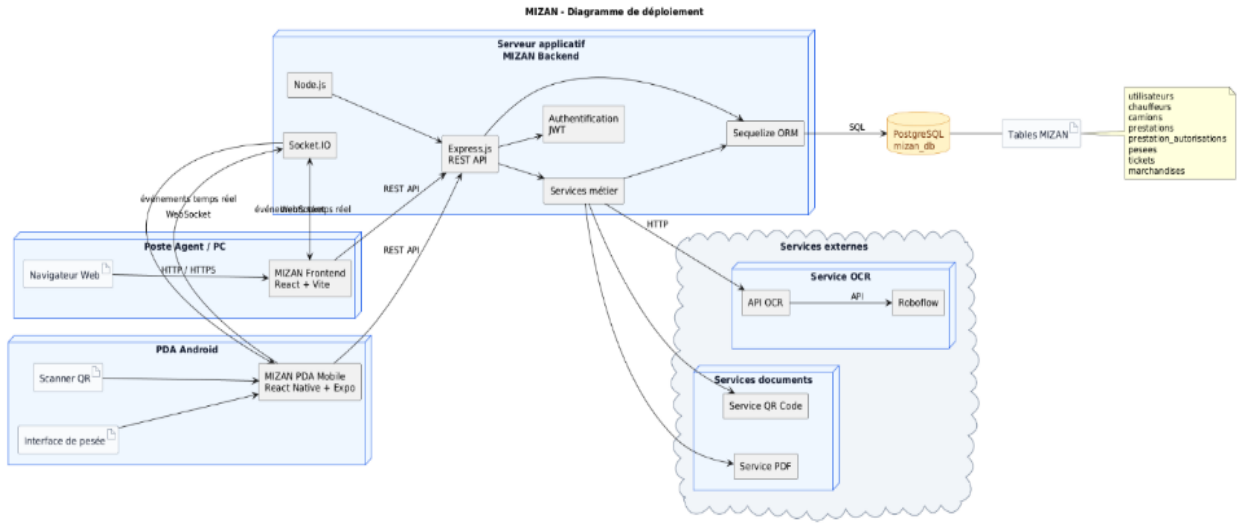


Figure 15 Diagramme de Deploiement

L'ensemble des composants est déployé au sein d'un réseau local représenté par le nuage LAN. Ce réseau interconnecte tous les éléments du système et assure la communication entre les différents nœuds. L'architecture suit une organisation en

couches avec une séparation claire entre les clients, le serveur applicatif, la base de données et les services externes.

La couche cliente comprend deux types de postes. Le premier est le Poste Agent ou PC, qui exécute un navigateur web chargé d'afficher l'application frontend développée avec React et Vite. Le navigateur communique avec le frontend via les protocoles HTTP ou HTTPS. Le second est le PDA Android, un terminal mobile utilisé sur le terrain pour les opérations de pesée. Il exécute l'application mobile développée avec React Native et Expo. Cette application intègre un composant de scanner QR pour l'identification des chauffeurs et une interface de pesée pour l'affichage des poids en temps réel.

Le serveur applicatif constitue le cœur du système. Il héberge plusieurs composants logiciels interconnectés. Node.js représente l'environnement d'exécution JavaScript côté serveur. Express.js constitue le framework web qui expose l'API REST permettant aux clients d'interagir avec le système. Socket.IO est le module de communication en temps réel qui établit des connexions WebSocket avec les clients pour diffuser les événements et les mises à jour instantanées. Les Services métier encapsulent la logique fonctionnelle du système, notamment l'identification des véhicules, la vérification des autorisations, le calcul des poids et la génération des tickets. Le composant Authentification JWT gère la sécurité et l'authentification des utilisateurs via des jetons JSON Web Token. Enfin, Sequelize ORM est la couche d'accès aux données qui permet au serveur d'interagir avec la base de données PostgreSQL en générant automatiquement les requêtes SQL.

La base de données PostgreSQL, nommée mizan_db, stocke l'ensemble des données persistantes du système. Elle contient plusieurs tables essentielles. La table utilisateurs stocke les comptes des agents et administrateurs. La table chauffeurs conserve les informations des chauffeurs enregistrés. La table camions contient les données des véhicules. La table prestations représente les opérations de transport. La table prestation_autorisations gère les autorisations d'accès. La table pesees enregistre toutes les opérations de pesée. La table tickets stocke les tickets générés. La table marchandises décrit les types de biens transportés. Le serveur applicatif communique avec la base de données via Sequelize en envoyant des requêtes SQL.

Le service OCR est un composant externe dédié à la reconnaissance optique de caractères. Il est déployé sur un serveur séparé et expose une API OCR accessible via HTTP. Ce service utilise la plateforme Roboflow pour exécuter des workflows de détection et de reconnaissance. Deux workflows distincts sont configurés, l'un pour la détection des plaques de tracteur et l'autre pour les plaques de remorque. Les services métier du serveur applicatif envoient les images capturées au service OCR via des requêtes HTTP et reçoivent en retour les matricules extraits accompagnés de scores de confiance.

Les services techniques regroupent deux composants spécialisés. Le Service QR Code est responsable de la génération et de la lecture des codes QR associés aux tickets et aux chauffeurs. Le Service PDF génère les documents PDF des tickets à partir des données

des pesées. Ces services sont invoqués par les services métier lors de la génération des tickets.

Les connexions entre les composants suivent des protocoles standardisés. Le frontend web communique avec l'API Express via des requêtes REST et avec Socket.IO via WebSocket pour recevoir les événements en temps réel. L'application mobile PDA communique également avec l'API Express via HTTP et avec Socket.IO via WebSocket. Le scanner QR et l'interface de pesée sont des composants internes au PDA qui interagissent directement avec l'application mobile.

Au sein du serveur, Node.js exécute Express, qui délègue les requêtes aux services métier et au module JWT pour l'authentification. Sequelize assure l'interface avec la base de données. Les services métier appellent le service OCR pour l'identification des véhicules et les services QR et PDF pour la génération des tickets. Socket.IO diffuse les événements temps réel vers le frontend web et l'application mobile, permettant ainsi la mise à jour instantanée des interfaces sans intervention manuelle.

3.2 Conception de la base de données

Dans le cadre du projet MIZAN, la base de données est utilisée pour centraliser les informations relatives aux utilisateurs, aux chauffeurs, aux camions, aux prestations, aux autorisations, aux opérations de pesage, aux tickets et aux marchandises. Elle assure également la conservation de l'historique des opérations et facilite la consultation des informations depuis l'application Web et l'application mobile PDA.

La base de données repose sur un modèle relationnel. Les données sont organisées dans plusieurs tables reliées entre elles à l'aide de clés primaires et de clés étrangères. Cette organisation permet d'éviter la duplication des informations et de maintenir l'intégrité des données.

3.2.1 Choix du modèle relationnel

Chaque table représente une entité du système. Par exemple, la table camions contient les informations relatives aux véhicules, tandis que la table pesees contient les informations concernant les opérations de pesage.

Le modèle relationnel présente plusieurs avantages dans le contexte du projet :

- organisation claire des données ;
- réduction de la redondance des informations ;
- possibilité d'utiliser des requêtes SQL ;
- compatibilité avec l'architecture backend basée sur Node.js, Express.js et Sequelize.

La base de données utilisée pour MIZAN est PostgreSQL. La communication entre le backend et la base de données est assurée par l'ORM Sequelize, qui permet de définir les modèles, les associations et les contraintes à partir du code JavaScript.

3.2.2 Modèle conceptuel de données

Le modèle conceptuel de données permet de représenter les principales entités du système ainsi que les relations entre elles, sans entrer directement dans les détails techniques de l'implémentation.

Dans le système MIZAN, les principales entités identifiées sont :

- Utilisateur ;
- Chauffeur ;
- Camion ;
- Prestation ;
- Autorisation de prestation ;
- Pesée ;
- Ticket ;
- Marchandise.

Chaque entité possède des informations propres appelées attributs. Par exemple, un camion est identifié par un matricule, un chauffeur possède une identité et une référence QR, tandis qu'une pesée contient les poids d'entrée, de sortie et le poids net.

Les principales relations conceptuelles sont les suivantes :

- un utilisateur peut être associé à un profil chauffeur ou agent ;
- un camion peut être concerné par plusieurs prestations ;
- une prestation peut être associée à un camion et à un chauffeur ;
- une prestation peut posséder plusieurs autorisations ;
- un camion peut effectuer plusieurs opérations de pesage ;
- un chauffeur peut être associé à plusieurs pesées ;
- une pesée peut être liée à une marchandise ;
- une pesée peut générer un ticket ;
- un ticket permet d'identifier et de suivre une opération de pesage.

3.2.3 Modèle logique de données

Le modèle logique de données traduit le modèle conceptuel sous forme de tables relationnelles. Chaque entité devient une table et chaque relation est représentée à l'aide de clés primaires et de clés étrangères.

Le modèle logique de la base de données MIZAN est organisé autour des tables suivantes :

Tableau 2 Modèle logique de la base de données

Table	Rôle
utilisateurs	Stocker les comptes utilisateurs et leurs informations d'authentification
chauffeurs	Stocker les informations relatives aux chauffeurs
camions	Stocker les informations relatives aux véhicules
prestations	Représenter les prestations de transport ou de manutention
prestation_autorisations	Gérer les autorisations d'accès à une prestation
pesees	Enregistrer les opérations de pesage d'entrée et de sortie
tickets	Conserver les tickets générés après une opération de pesage
marchandises	Stocker les informations sur les marchandises transportées

Les tables sont reliées afin de permettre le suivi complet d'une opération. Une pesée peut ainsi être associée à un camion, à un chauffeur, à une prestation, à une marchandise et à un agent. Cette organisation permet de retrouver toutes les informations liées à une opération donnée.

3.2.4 Description des principales tables

a. Table utilisateurs

La table utilisateurs contient les informations relatives aux personnes pouvant accéder au système MIZAN. Elle permet notamment de gérer l'authentification et les droits d'accès à l'application.

La clé primaire de cette table est id. Elle permet d'identifier chaque utilisateur de manière unique.

Les rôles peuvent notamment permettre de distinguer les profils suivants :

- administrateur ;
- agent de pesage ;
- autre profil autorisé selon les besoins de l'organisation.

Le mot de passe ne doit pas être enregistré en clair. Il doit être protégé à l'aide d'un mécanisme de hachage adapté.

b. Table chauffeurs

La table chauffeurs contient les informations relatives aux chauffeurs intervenant dans les opérations de pesage.

Elle permet de conserver l'identité du chauffeur et de faciliter sa vérification lors du passage au poste de pesage.

La clé primaire est id.

Le code QR permet d'identifier rapidement un chauffeur depuis l'application mobile PDA. La photo de référence peut également être utilisée dans le processus de vérification d'identité.

Dans le modèle conceptuel, un chauffeur peut être associé à plusieurs opérations de pesage. Cette relation permet de consulter l'historique des passages d'un chauffeur.

c. Table camions

La table camions représente les véhicules utilisés dans les opérations de transport.

Elle contient principalement les informations permettant d'identifier un camion à partir de son matricule.

La clé primaire est id.

Le matricule constitue une information importante dans le processus MIZAN. Il peut être saisi manuellement ou identifié automatiquement à partir d'une image grâce au service OCR.

Un camion peut être associé à plusieurs prestations et à plusieurs pesées. Cette relation permet de suivre l'historique des opérations effectuées par un véhicule.

d. Table prestations

La table prestations contient les informations relatives aux prestations prises en charge par le système MIZAN.

Une prestation représente une opération métier pour laquelle un camion doit accéder au poste de pesage.

La clé primaire est id.

La table prestations peut être liée à la table camions à travers le champ camion_id. Elle peut également être associée à un chauffeur à travers le champ chauffeur_id.

Cette association permet de retrouver la prestation correspondant à un camion identifié au niveau de la porte de pesage. Elle permet également à l'application PDA de consulter les prestations disponibles et d'effectuer les opérations autorisées.

Une prestation peut être associée à plusieurs opérations de pesage, notamment une pesée d'entrée et une pesée de sortie.

e. Table prestation_autorisations

La table prestation_autorisations permet de gérer les règles d'autorisation associées à une prestation.

Elle est utilisée pour déterminer si un camion est autorisé à accéder à une prestation donnée.

La clé primaire est id.

La clé étrangère prestation_id référence la table prestations. La clé étrangère camion_id référence la table camions.

Dans la version actuelle du projet, le contrôle est principalement basé sur l'autorisation du camion. Le champ type_autorisation permet de préciser la nature de l'autorisation, par exemple : CAMION

Le champ actif permet de savoir si l'autorisation est encore valide. Une autorisation inactive ne doit pas permettre l'accès à la prestation.

Cette table permet de séparer les informations de la prestation des règles d'accès qui lui sont associées. Elle rend ainsi le système plus flexible et facilite l'ajout de nouvelles règles d'autorisation.

f. Table pesees

La table pesees constitue l'une des tables principales du système MIZAN. Elle permet d'enregistrer chaque opération de pesage réalisée au niveau du poste de pesage.

Elle contient les informations nécessaires au suivi d'une pesée d'entrée, d'une pesée de sortie et au calcul du poids net.

La clé primaire est id.

Les clés étrangères permettent de relier la pesée aux différentes entités métier :

- camion_id vers camions ;
- chauffeur_id vers chauffeurs ;
- prestation_id vers prestations ;
- marchandise_id vers marchandises ;

Le champ statut permet de représenter l'état d'avancement de l'opération. Il peut notamment prendre des valeurs telles que :

- ENTREE ;
- SORTIE ;
- TERMINEE ;
- autre état défini par les règles métier.

La table pesees permet également de conserver l'historique des opérations et de retrouver les informations liées à un camion, un chauffeur ou une prestation.

g. Table tickets

La table tickets contient les tickets générés à la suite d'une opération de pesage.

Le ticket constitue une preuve ou un support de suivi de l'opération. Il peut être représenté sous forme numérique ou PDF et peut contenir un code QR utilisé par l'application PDA.

La clé primaire est id.

La clé étrangère pesee_id référence la table pesees.

Une pesée peut être associée à un ticket généré après l'enregistrement de l'opération. Le ticket permet notamment :

Le code QR du ticket permet de transmettre une référence unique entre l'application Web et l'application mobile.

h. Table marchandises

La table marchandises contient les informations relatives aux marchandises transportées ou concernées par les opérations de pesage.

Elle permet d'associer une marchandise à une pesée et de conserver les informations nécessaires au suivi de l'opération.

La clé primaire est id.

La table marchandises est liée à la table pesees par la clé étrangère marchandise_id.

Cette relation permet de savoir quelle marchandise était associée à une opération de pesage donnée.

3.2.5 Clés étrangères et relations

Une clé étrangère permet de créer une relation entre deux tables. Elle référence généralement la clé primaire d'une autre table.

Dans MIZAN, les principales clés étrangères sont les suivantes :

Tableau 3 Les principales clés étrangères

Table	Clé étrangère	Table référencée
prestations	camion_id	camions.id
prestations	chauffeur_id	chauffeurs.id
prestation_autorisations	prestation_id	prestations.id
prestation_autorisations	camion_id	camions.id
prestation_autorisations	chauffeur_id	chauffeurs.id, si utilisé
pesees	camion_id	camions.id
pesees	chauffeur_id	chauffeurs.id
pesees	prestation_id	prestations.id
pesees	marchandise_id	marchandises.id
pesees	agent_id	utilisateurs.id, si l'agent est un utilisateur
tickets	pesee_id	pesees.id

Ces relations permettent notamment de réaliser les opérations suivantes :

- retrouver les pesées d'un camion ;
- retrouver le chauffeur associé à une pesée ;
- consulter les prestations d'un camion ;
- vérifier les autorisations d'une prestation ;
- retrouver le ticket correspondant à une pesée ;
- consulter l'historique d'une marchandise.

3.2.6 Contraintes de la base de données

Les contraintes permettent de garantir que les données enregistrées respectent les règles définies par le système.

Les principales contraintes utilisées sont les suivantes :

a. Contrainte d'unicité

Certaines informations doivent être uniques afin d'éviter les doublons.

Exemples :

- identifiant utilisateur ;
- adresse e-mail ;
- matricule d'un camion, selon les règles retenues ;
- code QR d'un chauffeur ;
- code QR d'un ticket ;
- numéro de ticket.

Une contrainte UNIQUE peut être utilisée pour empêcher l'enregistrement de deux valeurs identiques dans une même colonne.

b. Contrainte de non-nullité

La contrainte NOT NULL permet d'imposer la présence d'une valeur.

Par exemple, certaines informations sont nécessaires pour enregistrer une pesée :

- camion concerné ;
- poids d'entrée ;
- statut de la pesée ;
- date de création.

Cependant, certains champs peuvent rester facultatifs selon le scénario métier. Par exemple, la prestation peut être renseignée ultérieurement dans le cas où l'affectation est réalisée depuis le PDA après le scan du ticket.

c. Contraintes de référence

Les clés étrangères empêchent l'association d'un enregistrement à une entité inexistante.

Par exemple, une pesée ne doit pas référencer un camion qui n'existe pas dans la table camions.

Cette contrainte garantit la cohérence entre les différentes tables.

d. Contraintes de domaine

Les contraintes de domaine permettent de contrôler les valeurs autorisées pour certains champs.

Exemples :

- un poids doit être positif ou nul ;
- un statut doit appartenir à une liste de valeurs définies ;
- un type d'autorisation doit être valide ;
- un identifiant doit respecter le type numérique attendu.

e. Contraintes temporelles

Les dates doivent respecter la logique du processus de pesage.

Par exemple :

- la date de sortie ne doit pas être antérieure à la date d'entrée ;
- une pesée terminée doit posséder les informations nécessaires ;
- un ticket doit être généré après l'enregistrement de la pesée.

Ces règles peuvent être contrôlées par la base de données ou par la logique métier du backend.

3.2.7 Intégrité des données

L'intégrité des données correspond à l'ensemble des mécanismes permettant de garantir que les informations enregistrées sont exactes, cohérentes et fiables.

Dans le système MIZAN, plusieurs types d'intégrité sont pris en compte.

Intégrité d'entité

Chaque enregistrement possède un identifiant unique grâce à la clé primaire id. Il n'est donc pas possible d'avoir deux lignes avec la même clé primaire.

Intégrité référentielle

Les clés étrangères garantissent que les relations entre les tables sont valides.

Par exemple :

- une pesée doit référencer un camion existant ;
- une autorisation doit référencer une prestation existante ;
- un ticket doit être associé à une pesée existante.

Intégrité de domaine

Les champs doivent respecter les types et les valeurs autorisées.

Par exemple :

- les poids sont enregistrés sous un type numérique ;
- les dates sont enregistrées dans un format adapté ;
- les statuts sont limités aux valeurs définies par le système ;
- les champs obligatoires ne peuvent pas être laissés vides.

Intégrité métier

L'intégrité métier correspond aux règles spécifiques au fonctionnement de MIZAN.

Exemples :

- un camion doit être identifié avant l'enregistrement de la pesée ;
- le poids doit être stable avant sa validation ;
- une autorisation active doit être vérifiée avant l'accès à une prestation ;
- le poids net est calculé à partir des poids disponibles ;
- une pesée de sortie doit être liée à une opération d'entrée ;
- un ticket doit être associé à la pesée correspondante ;
- une opération terminée doit conserver son historique.

Ces contrôles sont répartis entre la base de données, le backend et les interfaces Web et mobile.

3.2.8 Schéma simplifié des relations

Le schéma logique simplifié peut être représenté de la manière suivante :

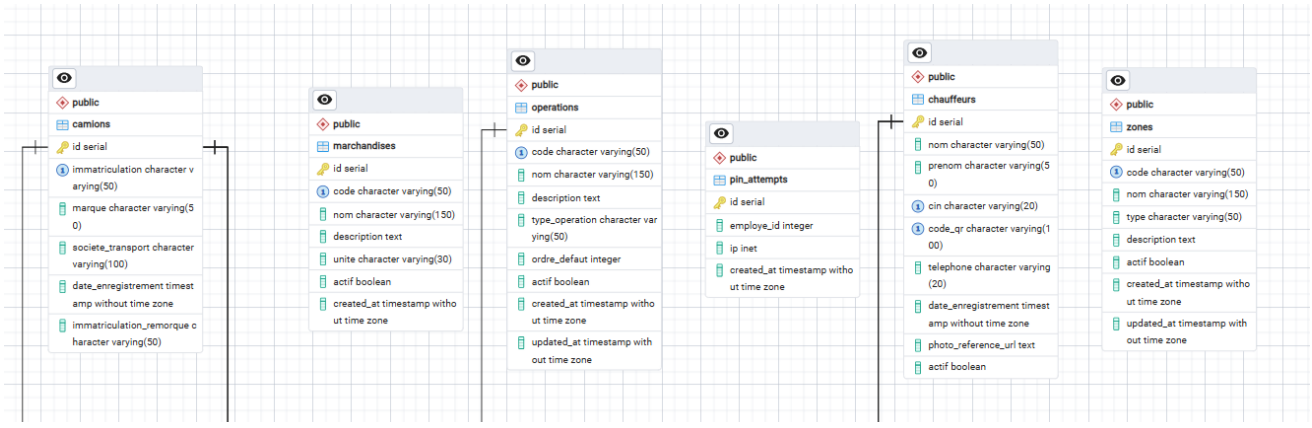


Figure 16 Tableaux db part 1

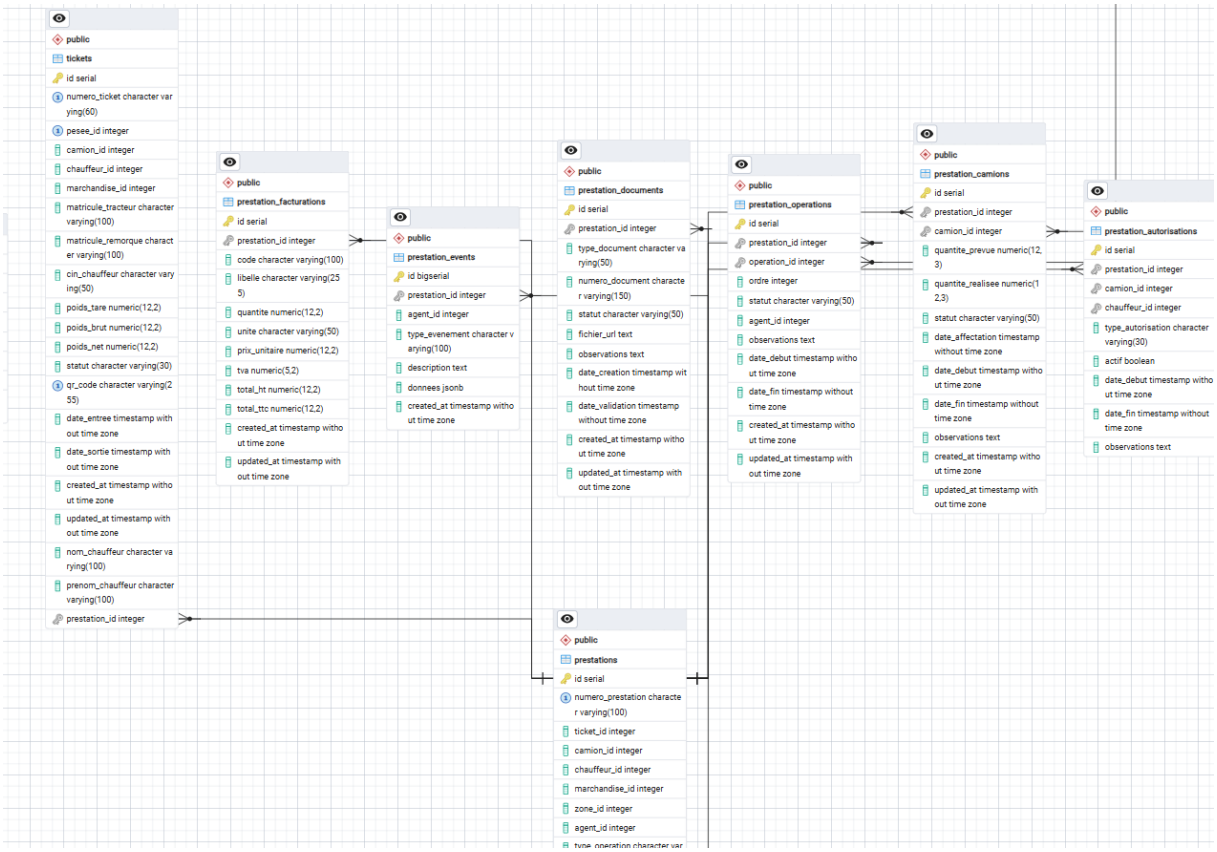


Figure 17 Tableaux db part 2

3.2.9 Conclusion

La conception de la base de données a permis de définir une structure cohérente pour les informations manipulées par le système MIZAN. Le modèle relationnel repose sur plusieurs tables principales représentant les utilisateurs, les chauffeurs, les camions, les prestations, les autorisations, les pesées, les tickets et les marchandises.

L'utilisation des clés primaires, des clés étrangères et des contraintes permet de garantir l'unicité, la cohérence et l'intégrité des données. La table pesees occupe une position centrale, car elle assure le suivi des opérations de pesage et relie les différentes entités métier.

Cette conception constitue une base importante pour l'implémentation du backend, la création des modèles Sequelize et le développement des fonctionnalités Web et mobile. Elle facilite également l'évolution future du système, notamment l'ajout de nouvelles règles d'autorisation, de nouveaux profils utilisateurs ou de nouvelles fonctionnalités de suivi.

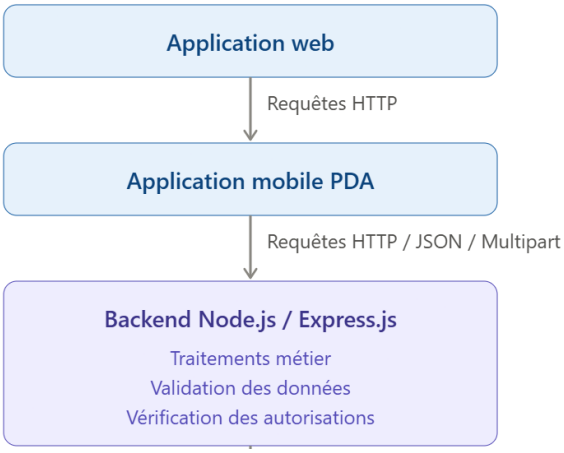
3.3 Conception des API

Dans le projet MIZAN, l'application Web de supervision et l'application mobile PDA communiquent avec le backend à travers des API REST développées avec Node.js et Express.js. Le backend joue le rôle d'intermédiaire entre les interfaces utilisateurs, les services externes et la base de données PostgreSQL.

Cette organisation permet de centraliser les traitements métier, de sécuriser les accès aux données et d'éviter que les applications clientes communiquent directement avec la base de données.

3.3.1 Architecture générale des API

Le fonctionnement général des échanges est organisé de la manière suivante :



3.3.2 Principales routes de l'API

Les principales routes utilisées dans le système MIZAN sont présentées dans le tableau suivant.

Tableau 4 Principales routes de l'API

Module	Exemple de route	Fonction
Authentification	POST /api/auth/login	Authentifier un utilisateur
Authentification	POST /api/auth/register	Créer un compte utilisateur, si activé
Camions	GET /api/camions	Consulter la liste des camions
Camions	POST /api/camions	Ajouter un camion

Camions	GET /api/camions/:id	Consulter un camion précis
Camions	PUT /api/camions/:id	Modifier les informations d'un camion
Camions	DELETE /api/camions/:id	Supprimer un camion, selon les droits
Chauffeurs	GET /api/chauffeurs	Consulter la liste des chauffeurs
Chauffeurs	GET /api/chauffeurs/qr/:qr	Rechercher un chauffeur à partir de son QR Code
Chauffeurs	POST /api/chauffeurs/enregistrer-avec-qr	Enregistrer un chauffeur avec un QR Code
Chauffeurs	POST /api/chauffeurs/:id/photo-reference	Enregistrer ou modifier une photo de référence
Prestations	GET /api/prestations	Consulter les prestations
Prestations	POST /api/prestations	Créer une prestation
Prestations	GET /api/prestations/:id	Consulter une prestation précise
Prestations	GET /api/prestations/qr/:qrCode	Rechercher une prestation à partir d'un QR Code
Prestations	GET /api/prestations/pda	Consulter les prestations destinées au PDA
Prestations	PUT /api/prestations/:id	Modifier une prestation
Autorisations	POST /api/prestation-autorisations/valider	Vérifier l'autorisation d'un camion pour une prestation
Autorisations	GET /api/prestation-autorisations	Consulter les autorisations
Autorisations	POST /api/prestation-autorisations	Créer une autorisation
Autorisations	PUT /api/prestation-autorisations/:id	Modifier une autorisation
Pesée	POST /api/pesee/entree	Enregistrer une pesée d'entrée
Pesée	POST /api/pesee/sortie	Enregistrer une pesée de sortie

Pesée	GET /api/pesee	Consulter les opérations de pesage
Pesée	GET /api/pesee/:id	Consulter une pesée précise
Tickets	GET /api/tickets/pesee/:peseeId/pdf	Générer le ticket PDF d'une pesée
Tickets	GET /api/tickets/:id	Consulter les informations d'un ticket
Marchandises	GET /api/marchandises	Consulter les marchandises
Marchandises	POST /api/marchandises	Ajouter une marchandise
Marchandises	PUT /api/marchandises/:id	Modifier une marchandise

Les routes présentées correspondent aux principales fonctionnalités du système. Certaines routes complémentaires peuvent être ajoutées selon les besoins de l'application et l'évolution du projet.

3.3.3 Conclusion de la conception des API

La conception des API a permis de définir une interface de communication claire entre les applications Web, mobile et le backend du système MIZAN.

Les routes REST sont organisées par modules fonctionnels afin de faciliter la maintenance et l'évolution de l'application. Elles permettent notamment de gérer l'authentification, les camions, les chauffeurs, les prestations, les autorisations, les pesées, les tickets et les marchandises.

L'utilisation de JSON, des méthodes HTTP et de codes de réponse standardisés facilite les échanges entre les composants. La communication en temps réel avec Socket.IO complète les API REST en permettant la mise à jour instantanée du poids et des statuts.

Cette conception constitue une base technique pour l'implémentation du backend et pour l'intégration des interfaces Web et mobile.

3.4 Conclusion

Ce chapitre a présenté la conception générale et l'architecture technique du système MIZAN.

La première partie a permis de décrire l'architecture globale de la solution, composée d'une application Web de supervision, d'une application mobile PDA, d'un backend

Node.js/Express.js, d'une base de données PostgreSQL et de plusieurs services externes, notamment le service OCR, la génération des QR Codes et la génération des tickets PDF.

La modélisation UML a ensuite permis de représenter les acteurs, les fonctionnalités, les classes, les interactions et les différents états du processus de pesage. Ces diagrammes facilitent la compréhension du système et servent de référence pour les étapes de développement.

La conception de la base de données a défini les principales tables, leurs clés primaires, leurs clés étrangères et leurs relations. Elle permet d'assurer la centralisation, la cohérence et la traçabilité des informations relatives aux camions, chauffeurs, prestations, pesées et tickets.

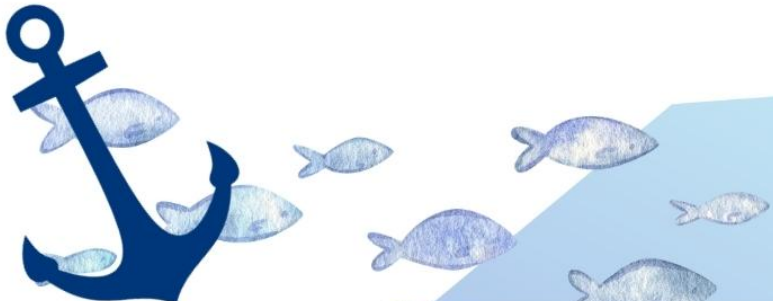
Enfin, la conception des API a précisé les principales routes utilisées pour assurer la communication entre les applications clientes et le backend. Cette organisation permet de centraliser les règles métier, de contrôler les autorisations et de faciliter l'intégration des différents composants.

L'ensemble de ces travaux de conception constitue une base solide pour la phase suivante, consacrée à la réalisation et à l'implémentation du système MIZAN.



Chapitre 4

RÉALISATION ET IMPLÉMENTATION





4.1 Introduction

Après avoir présenté les besoins fonctionnels, l'architecture technique et la conception du système MIZAN, ce chapitre est consacré à la phase de réalisation et d'implémentation de la solution. Cette étape consiste à transformer les choix de conception en fonctionnalités concrètes et opérationnelles.

La phase de réalisation comprend la mise en place de l'environnement de développement, l'organisation des différents projets, l'implémentation du backend, le développement de l'interface web et l'intégration du service de reconnaissance automatique des plaques d'immatriculation.

Le système MIZAN est composé de plusieurs éléments complémentaires : un backend développé avec Node.js et Express.js, une base de données PostgreSQL, une application web de supervision, un service OCR pour l'identification des véhicules, un système de génération de tickets PDF et de QR Codes, un mécanisme de communication en temps réel avec Socket.IO et une application mobile PDA utilisée pour certaines opérations de suivi et de contrôle.

L'objectif de cette phase est de mettre en œuvre une solution fonctionnelle permettant de gérer les véhicules, les chauffeurs, les prestations, les autorisations et les opérations de pesage, tout en assurant la traçabilité complète des informations.

4.2 Environnement de développement

La réalisation du projet MIZAN a nécessité l'utilisation de plusieurs outils logiciels et technologies. Chaque outil intervient dans une partie précise du développement, du stockage des données, des tests ou de l'intégration des services externes.

4.2.1 Antigravity IDE

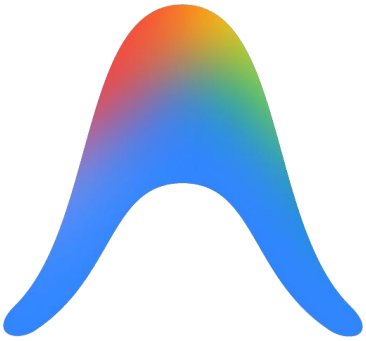


Figure 18 Antigravity

Antigravity a été utilisé comme environnement principal de développement. Cet éditeur de code source permet de créer et modifier les fichiers du projet, d'organiser les dossiers du backend et du frontend, de développer les composants React, d'écrire les contrôleurs, modèles et routes Express, de consulter les erreurs dans le terminal, d'installer des extensions adaptées aux technologies utilisées et d'effectuer le débogage du code.

4.2.2 Node.js



Figure 19 Node.js

Node.js est un environnement d'exécution JavaScript côté serveur qui permet d'exécuter le code backend en dehors du navigateur. Dans le projet MIZAN, Node.js est utilisé pour créer le serveur backend, traiter les requêtes HTTP, communiquer avec PostgreSQL, gérer les utilisateurs, traiter les opérations de pesage, gérer les prestations et les autorisations, générer les tickets, communiquer avec les services externes et transmettre les événements en temps réel. Le backend du projet est exécuté localement sur le port 5000.

4.2.3 npm



Figure 20 npm

npm (Node Package Manager) est le gestionnaire de paquets utilisé avec Node.js. Il permet d'installer et de gérer les bibliothèques nécessaires au projet. Parmi les dépendances utilisées dans MIZAN, on peut citer express pour la création du serveur, sequelize pour l'accès à la base de données, pg pour la communication avec PostgreSQL, cors pour autoriser les communications entre les applications, jsonwebtoken pour la gestion des tokens JWT, socket.io pour la communication en temps réel, qrcode pour la génération des QR Codes, les bibliothèques de génération de fichiers PDF et les bibliothèques React côté frontend. Le fichier package.json contient les informations du projet ainsi que les dépendances nécessaires à son fonctionnement.

4.2.4 PostgreSQL



Figure 21 PostgreSQL

PostgreSQL est le système de gestion de base de données relationnelle utilisé pour le projet MIZAN. La base de données utilisée dans l'environnement de développement est nommée mizan_db. PostgreSQL assure la conservation des données et permet d'appliquer différentes contraintes, notamment les clés primaires, les clés étrangères et les contraintes d'intégrité.

4.2.5 Git et GitHub



Figure 22 Github

Git est un système de gestion de versions permettant de suivre les modifications effectuées dans le code source. Lorsqu'il est utilisé dans le projet, il permet de conserver l'historique des modifications, de revenir à une version précédente, de créer des branches de développement, de comparer différentes versions du code et de faciliter la sauvegarde du projet. GitHub peut être utilisé comme plateforme distante pour stocker le code source et faciliter son partage. La gestion de versions est particulièrement utile dans un projet composé de plusieurs applications et de nombreux fichiers.

4.2.8 Postman



Figure 23 Postman

Les API ont été testées à l'aide de requêtes HTTP exécutées depuis le navigateur, des scripts Node.js ou un outil de test d'API tel que Postman. Ces outils permettent de vérifier les méthodes HTTP, les paramètres envoyés, les réponses JSON, les codes HTTP, les erreurs de validation, les routes protégées et les opérations de création et de modification. Les tests ont porté notamment sur les routes relatives aux camions, chauffeurs, prestations, autorisations, pesées et tickets.

4.2.9 Roboflow



Figure 24 Roboflow

Roboflow a été utilisé pour intégrer le service de reconnaissance automatique des plaques d'immatriculation. Deux workflows OCR ont été utilisés : un workflow destiné à l'identification du tracteur et un workflow destiné à l'identification de la remorque. Les workflows utilisés sont moroccan-license-plate-ocr-1788096205153 et lecture-plaques-remorques-maroc-1788177 816517. Le service reçoit une image et retourne une prédiction contenant les informations détectées. Le backend ou le frontend traite ensuite le résultat afin d'extraire la matricule et de rechercher le véhicule dans la base de données.

4.2.10 Expo

Expo a été utilisé pour tester l'application mobile PDA développée avec React Native. Expo permet de lancer l'application mobile en mode développement, de tester les écrans sur un appareil physique, d'utiliser la caméra du téléphone, de scanner les QR



Figure 25 Expo Go

Codes, de vérifier la communication avec le backend et de tester les fonctionnalités de consultation et de validation. L'application PDA communique avec le backend à travers l'adresse IP lo cale de la machine de développement.

Le projet MIZAN est organisé en plusieurs applications et modules. Cette séparation permet de distinguer les responsabilités du backend, du frontend web et de l'application mobile. Les deux principaux projets étudiés dans ce chapitre sont mizan-backend et mizan-frontend. L'application mobile PDA est présentée plus en détail dans le chapitre 5.

4.2.11 Docker

Docker a été utilisé pour conteneuriser le serveur PostgreSQL du projet MIZAN. La conteneurisation permet d'exécuter la base de données dans un environnement isolé et indépendant de l'installation locale du système.

Dans le projet, Docker est principalement utilisé pour :

exécuter PostgreSQL dans un conteneur ;
isoler le serveur de base de données ;
faciliter le démarrage et l'arrêt de PostgreSQL ;
conserver les données à l'aide d'un volume ;
reproduire plus facilement l'environnement de développement ;
préparer le projet à un déploiement sur un environnement serveur.

4.3 Organisation du backend

Le backend contient la logique métier du système. Il reçoit les requêtes des applications clientes, applique les contrôles nécessaires, communique avec la base de données et retourne les réponses. L'organisation générale du backend est structurée autour de plusieurs dossiers spécialisés.

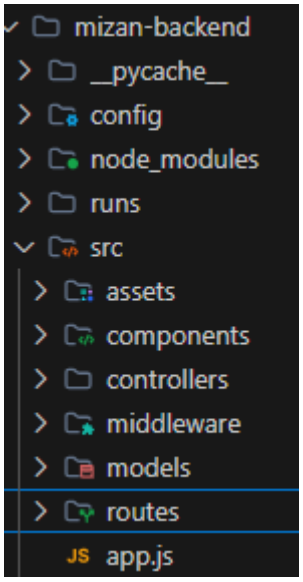


Figure 26 Organisation Backend

Le dossier controllers contient les fonctions responsables du traitement des requêtes. Les contrôleurs permettent de recevoir les paramètres envoyés par le frontend, d'appeler les modèles ou les services, d'appliquer les règles métier et de retourner une réponse HTTP. Les exemples de contrôleurs incluent le contrôleur d'authentification, le contrôleur des camions, le contrôleur des chauffeurs, le contrôleur des prestations, le contrôleur des pesées et le contrôleur des tickets.

Le dossier models représente les tables de la base de données. Les modèles sont définis avec Sequelize et permettent de manipuler les entités suivantes : utilisateur, chauffeur, camion, prestation, autorisation, pesée, ticket et marchandise. Les associations entre les modèles permettent de représenter les relations définies lors de la conception de la base de données.

Le dossier routes définit les URL accessibles par les applications clientes. Les exemples incluent /api/auth, /api/camions, /api/chauffeurs, /api/prestations, /api/prestation-autorisations, /api/pesee, /api/tickets et /api/marchandises. Chaque groupe de routes est associé à un contrôleur spécialisé.

Le dossier middlewares exécute des traitements intermédiaires avant l'arrivée de la requête au contrôleur. Ils peuvent être utilisés pour vérifier le token JWT, contrôler les droits d'accès, valider les données reçues, gérer les fichiers envoyés et traiter les erreurs.

Le dossier config contient les éléments de configuration du projet, notamment la connexion à PostgreSQL, les paramètres de l'environnement, les variables de configuration et les paramètres des services externes.

Le fichier server.js constitue le point d'entrée du backend. Il permet de créer l'application Express, de configurer les middlewares, de connecter la base de données, d'enregistrer les routes, d'initialiser Socket.IO et de démarrer le serveur.

4.4 Organisation du frontend web

Le frontend web contient les composants graphiques et les pages accessibles aux utilisateurs. Son organisation générale est structurée autour de plusieurs dossiers spécialisés.

Le dossier components contient les composants React réutilisables. Les exemples incluent le formulaire de saisie des matricules, le composant de vérification du chauffeur, le composant de pesée, le tableau de données, la fenêtre de confirmation et le composant de notification.

Le dossier pages contient les principales pages de l'application web. Il peut notamment regrouper la page de connexion, le tableau de bord, la page des camions, la page des chauffeurs, la page des prestations, la page des marchandises, la page d'historique et la page de consultation des tickets.

Le dossier services regroupe les fonctions de communication avec le backend. Ils permettent notamment d'envoyer des requêtes HTTP, de récupérer les camions, de consulter les prestations, de rechercher un chauffeur, d'enregistrer une pesée et de récupérer un ticket PDF.

Le dossier assets contient les ressources utilisées par l'interface, notamment les images, les icônes, les logos, les fichiers graphiques et les ressources de style.

Le fichier App.jsx constitue le composant principal de l'application React. Il permet d'organiser les pages, les composants et la navigation générale. Le fichier vite.config.js contient la configuration de Vite, utilisé comme outil de développement et de construction du frontend.

4.5 Implémentation du backend

4.5.1 Configuration du serveur

Le backend MIZAN a été développé avec Node.js et Express.js. Express permet de créer un serveur HTTP et de définir les routes utilisées par les applications web et mobile. Le serveur est configuré pour écouter sur le port 5000.

L'initialisation du serveur comprend plusieurs étapes : création de l'application Express, activation des middlewares, configuration du CORS, connexion à PostgreSQL, enregistrement des routes, initialisation de Socket.IO et démarrage du serveur. Le CORS permet d'autoriser les communications entre le frontend web, l'application mobile et le backend lorsque ces applications sont exécutées sur des adresses ou des ports différents.

Les données envoyées au serveur peuvent être transmises sous forme JSON, de paramètres URL, de paramètres de requête ou de données multipart/form-data. Le format multipart/form-data est particulièrement utile lors de l'envoi de photos ou de fichiers.

4.5.2 Connexion à PostgreSQL



Figure 27 Squeelize

La communication avec PostgreSQL est réalisée à l'aide de Sequelize. Sequelize permet de définir les modèles, de représenter les tables, de déclarer les associations, d'exécuter des requêtes, d'insérer des données, de modifier des enregistrements, de rechercher des informations et de gérer certaines contraintes. La connexion utilise la base de données mizan_db. Le backend doit vérifier que la connexion à la base de données est correctement établie avant de démarrer les traitements métier. En cas d'erreur de connexion, le serveur doit afficher un message explicatif afin de faciliter le diagnostic.

4.5.3 Authentification

Le module d'authentification permet de contrôler l'accès au système. Les principales fonctionnalités sont l'inscription d'un utilisateur, la connexion, la génération d'un token JWT, la vérification du token, la protection des routes et la déconnexion côté application.

Lors de la connexion, l'utilisateur transmet ses informations d'identification au backend. Le serveur vérifie les informations reçues puis retourne une réponse adaptée. Le JWT permet de représenter une session authentifiée. Lorsqu'une route est protégée, le backend vérifie la validité du token avant d'autoriser l'accès. Les routes sensibles peuvent ainsi être réservées aux utilisateurs authentifiés.

4.5 Implémentation du frontend web

L'application web MIZAN a été développée avec React. Elle constitue l'interface principale de supervision et permet aux utilisateurs autorisés d'accéder aux différentes fonctionnalités du système. L'interface a été conçue autour d'une présentation claire et adaptée à un environnement de supervision. Elle utilise une organisation par pages et composants afin de faciliter la navigation et la maintenance.

4.5.1 Page de connexion

La page de connexion permet à l'utilisateur de saisir ses informations d'authentification. Elle comprend généralement un champ identifiant ou e-mail, un champ mot de passe, un bouton de connexion, des messages d'erreur et une indication de l'état de chargement. Après validation, les informations sont envoyées au backend. Si l'authentification réussit, l'utilisateur est redirigé vers les fonctionnalités auxquelles son rôle lui donne accès.

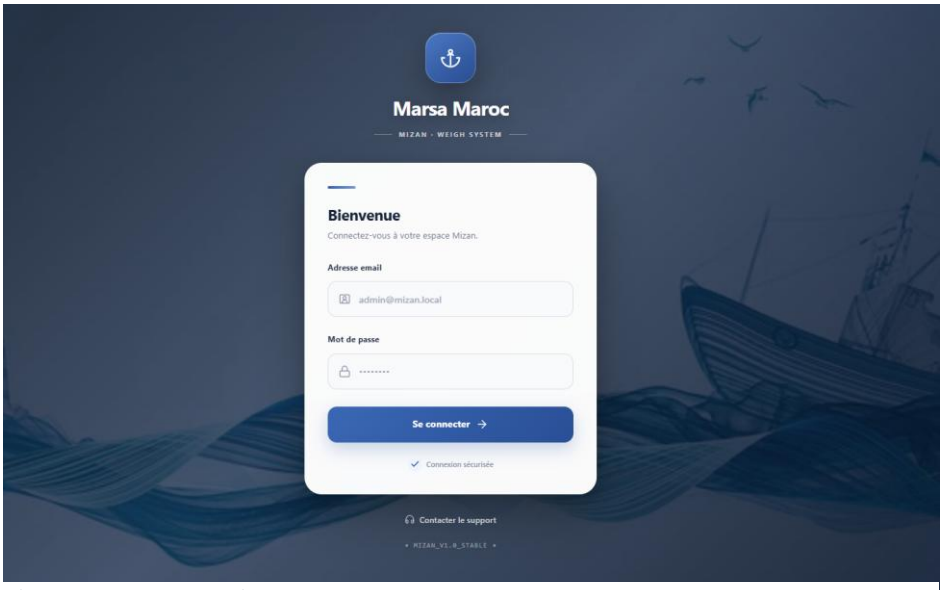


Figure 28 Page Connexion

4.5.2 Tableau de bord

Le tableau de bord représente l'écran principal de supervision. Il permet de consulter rapidement les opérations de pesage, les entrées enregistrées, les sorties enregistrées, les prestations, les statuts des opérations, les informations des camions et les mises à jour en temps réel.

L'en-tête de l'interface peut présenter les informations suivantes : MARSA MAROC, GATE DE PESAGE, Centre de supervision. Le tableau de bord utilise les données retournées par les API et peut recevoir des mises à jour via Socket.IO.

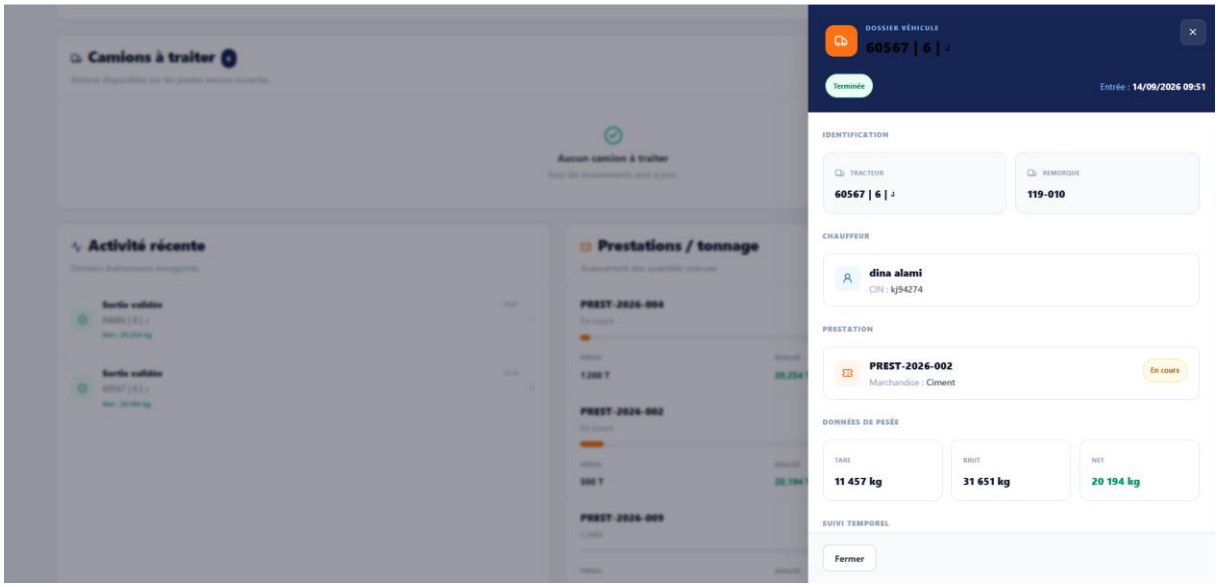


Figure 29 Tableau de bord

4.5.3 Gestion des chauffeurs

La page de gestion des chauffeurs permet de consulter et de gérer les informations d'identité. Elle peut afficher le nom et prénom, le numéro de CIN, le code QR, la photo de référence, le statut du chauffeur et l'historique des opérations. Cette interface facilite la vérification des informations avant ou après une opération de pesage.

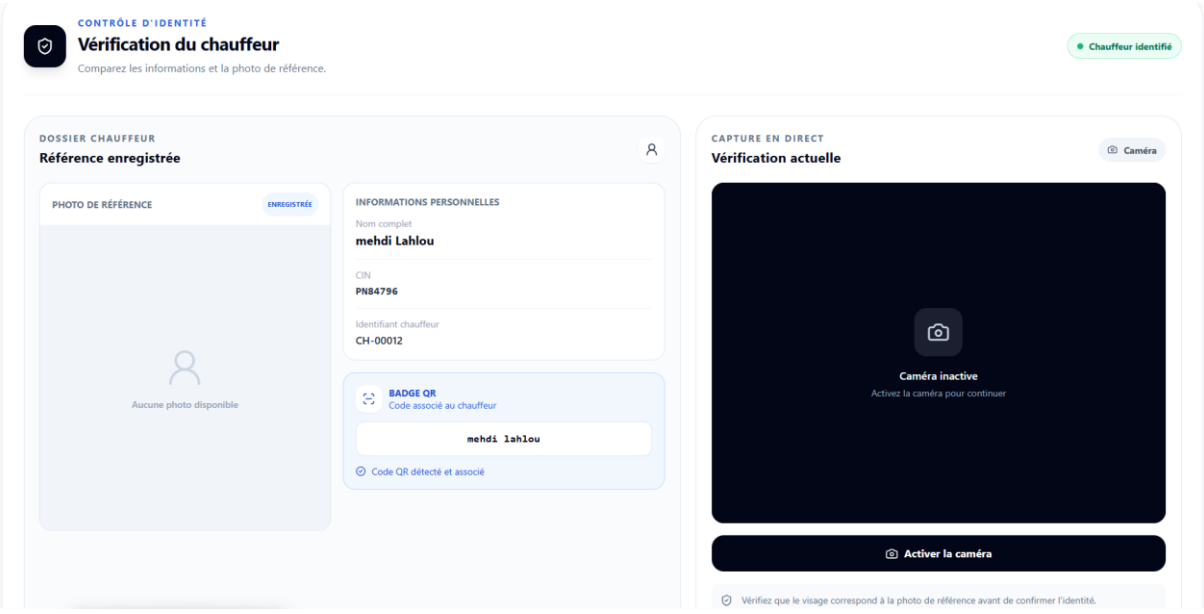


Figure 30 Infos Chauffeur

4.5.4 Écran de pesée

L'écran de pesée guide l'utilisateur à travers les différentes étapes du processus. Le workflow web comprend notamment l'identification du tracteur, l'identification de la remorque, la vérification du chauffeur, la validation de l'identité, la lecture du poids, la vérification de la stabilité, l'enregistrement de la pesée et la génération du ticket. L'écran affiche les informations nécessaires à chaque étape et permet de signaler les erreurs avant la validation.

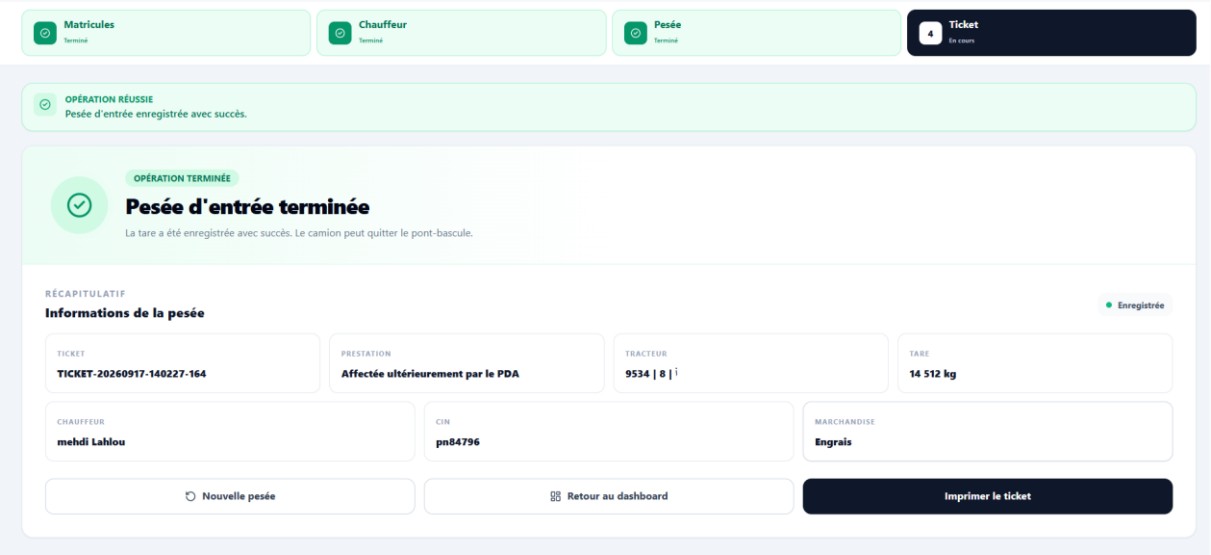


Figure 31 Ecran Pessage

4.5.5 Identification OCR

L'interface web intègre un composant permettant de capturer ou de sélectionner une image du véhicule. L'image est ensuite envoyée au service OCR. Le résultat est traité pour extraire la matricule. Le système peut afficher l'image capturée, la matricule détectée, le

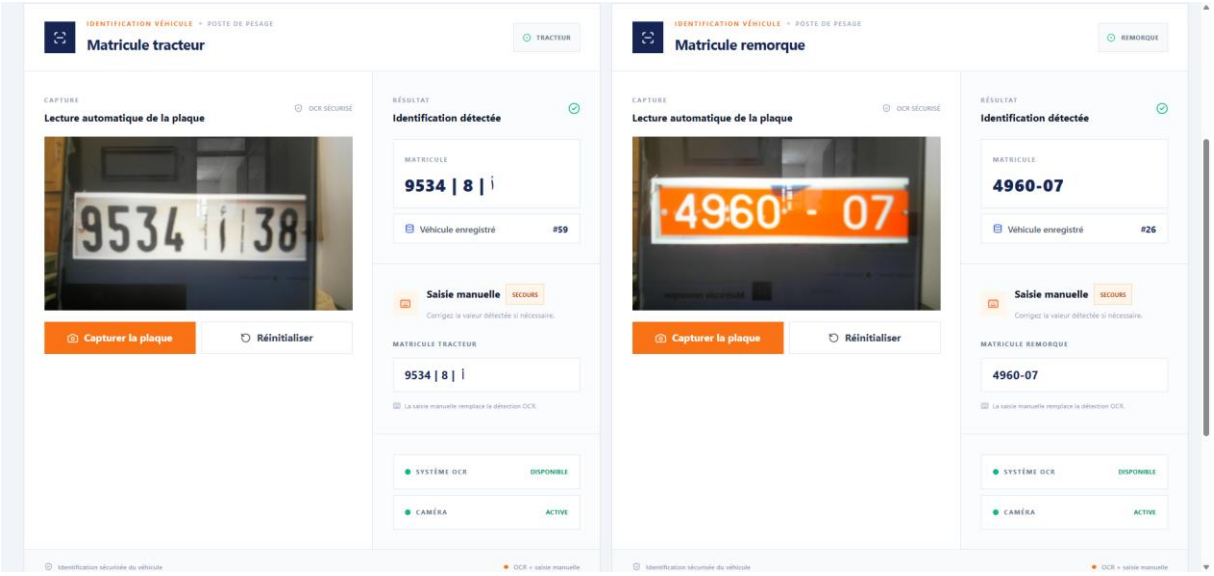


Figure 32 Detection Matricule

type de véhicule, le niveau de confiance si disponible, un message en cas d'échec et une possibilité de correction manuelle. L'identification OCR permet de réduire la saisie manuelle et d'accélérer le traitement des véhicules.

4.5.6 Consultation des tickets

L'application web permet de consulter les tickets associés aux pesées. L'utilisateur peut notamment afficher les informations du ticket, consulter le numéro, retrouver le code QR, ouvrir le document PDF, vérifier les poids enregistrés et retrouver la pesée correspondante. Cette fonctionnalité facilite l'archivage et la recherche des documents de suivi.

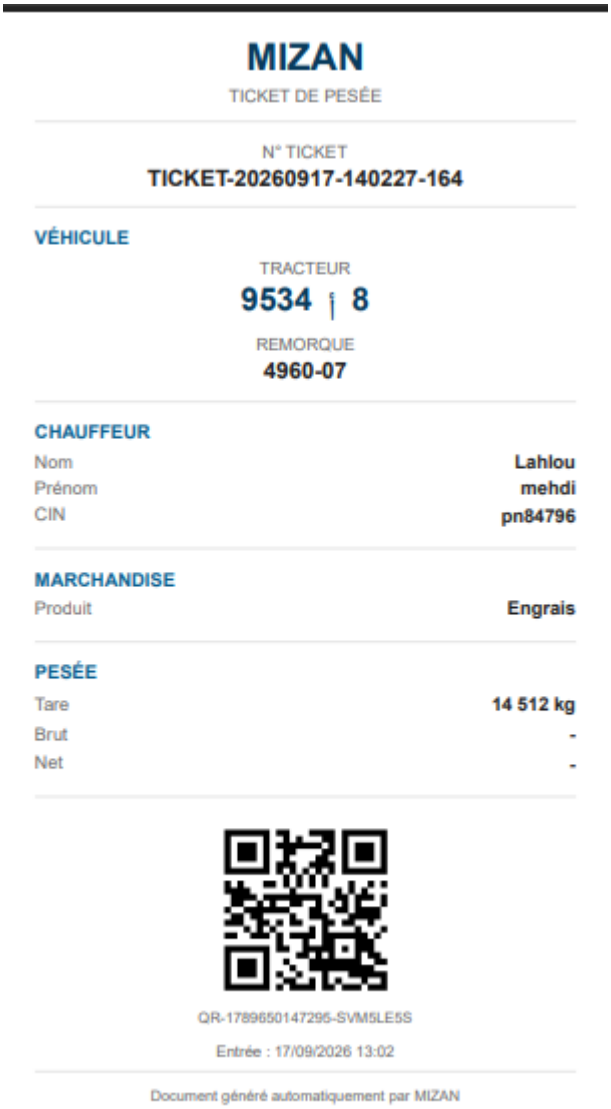


Figure 33 Ticket d'Entrée

4.6 Intégration du service OCR

4.6.1 Présentation du service OCR

Le système MIZAN intègre un service OCR permettant de reconnaître automatiquement les plaques d'immatriculation à partir d'une image. OCR signifie Optical Character Recognition, ou reconnaissance optique de caractères. Dans le projet, le service est utilisé pour identifier la matricule du tracteur et la matricule de la remorque. L'objectif est de réduire la saisie manuelle et de faciliter l'identification des véhicules au niveau du poste de pesage.

4.6.2 Traitement des erreurs OCR

La reconnaissance peut échouer dans certaines situations. Les principales erreurs possibles sont l'absence de plaque détectée, un texte incomplet, des caractères mal reconnus, une image trop sombre, une image floue, un mauvais workflow sélectionné ou une matricule absente de la base.

Dans ces cas, l'interface affiche un message d'erreur et peut proposer une saisie manuelle. Cette possibilité est importante afin de ne pas bloquer complètement le processus de pesage lorsqu'une reconnaissance automatique n'est pas fiable.

4.7 Fonctionnement Roboflow

4.7.1 Traitement de Workflow

Le traitement repose sur une architecture en plusieurs étapes : détection → redressement → découpage → reconnaissance → validation. Cette séparation est importante, car lire toute la plaque avec un seul OCR provoquait des confusions entre les chiffres, les barres | et les lettres arabes.

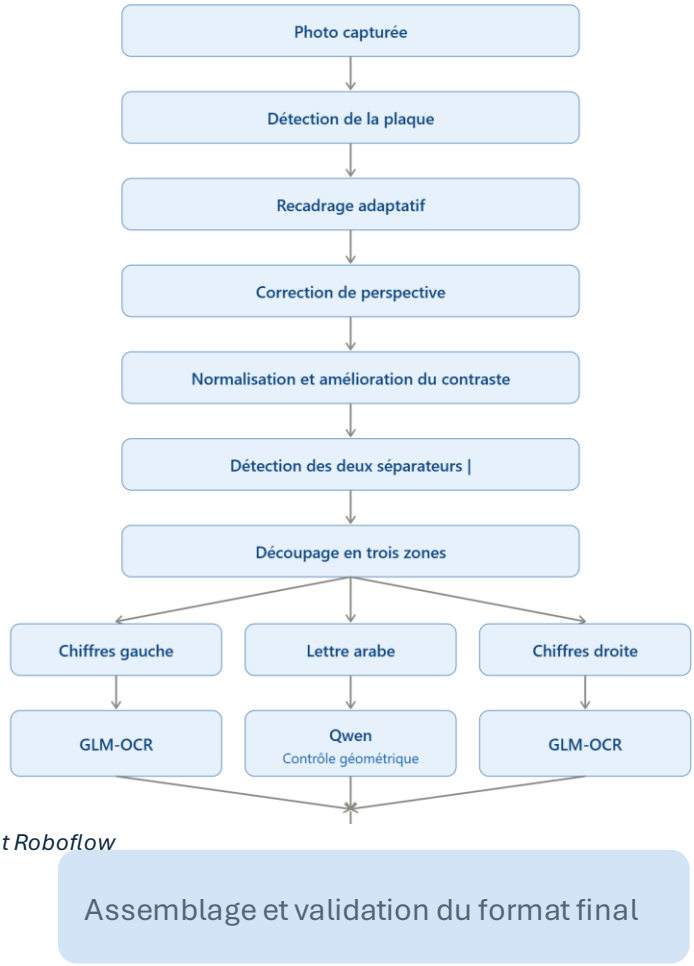


Figure 34 Fonctionnement Roboflow

4.7.2 Capture et envoi de l'image

Lorsque l'utilisateur clique sur le bouton de capture, l'application prend une photo de la caméra et l'envoie à l'API FastAPI.

1. L'API : lit l'image reçue ;
2. applique son orientation EXIF ;
3. la convertit au format RGB ;
4. conserve sa résolution d'origine ;
5. l'envoie au Workflow Roboflow.
6. result = client.run_workflow(

```
    workspace_name="chaimaala",
    workflow_id="moroccan-license-plate-ocr-1788096205153",
    images={"image": image},
    use_cache=False,
)
```

Le cache est désactivé afin que chaque nouvelle capture soit réellement analysée.

4.7.3 Détection de la plaque

La première étape du Workflow utilise un modèle de détection pour localiser la plaque dans l'image complète.

Le résultat est une boîte englobante contenant notamment :

- les coordonnées de la plaque ;
- sa largeur et sa hauteur ;
- la confiance de détection.

Par exemple :

Classe : License-Plate

Confiance : 91 %

Position : x, y

Dimensions : largeur × hauteur

Cette étape permet de trouver la plaque où qu'elle soit dans l'image. Elle ne dépend donc pas du cadre vert affiché dans l'application.

Le cadre vert peut aider l'utilisateur à bien viser, mais il n'est pas utilisé comme zone fixe de traitement.

4.7.4 Recherche des contours et correction de perspective

Une plaque capturée par une caméra n'est pas toujours parfaitement horizontale. Elle peut être inclinée ou observée en perspective.

Le Workflow applique plusieurs opérations OpenCV :

- conversion en niveaux de gris ;
- filtre bilatéral pour réduire le bruit tout en conservant les bords ;
- détection des contours avec Canny ;
- seuillage automatique d'Otsu ;
- opérations morphologiques pour reconnecter les bords ;
- recherche d'une forme rectangulaire correspondant à la plaque.

Les formes candidates sont évaluées selon :

- leur surface ;
- leur rapport largeur/hauteur ;
- leur rectangularité ;

- leur proximité avec le centre de la boîte détectée.

Les quatre coins de la meilleure forme sont ensuite ordonnés :

- coin supérieur gauche
- coin supérieur droit
- coin inférieur droit
- coin inférieur gauche

4.7.6 Amélioration de l'image

La plaque redressée passe ensuite par une amélioration modérée du contraste.

Le but est de mieux distinguer :

- les caractères noirs ;
- le fond blanc ;
- les barres verticales ;
- les petits signes comme la hamza de أ ou les points de certaines lettres.

L'amélioration reste modérée afin de ne pas effacer les petits détails. Un traitement trop agressif pourrait, par exemple, supprimer les points de ب ou la hamza de أ.

4.7.8 Lecture des groupes numériques

Les chiffres à gauche et à droite sont lus séparément avec GLM-OCR.

Zone gauche le modèle doit retourner uniquement une suite de chiffres : 74467

Zone droite le modèle doit retourner uniquement un ou deux chiffres :8

Le prompt précise explicitement que :

Une barre verticale n'est jamais le chiffre 1.

Cette séparation améliore la précision, car l'OCR ne doit plus comprendre toute la structure de la plaque en une seule fois.

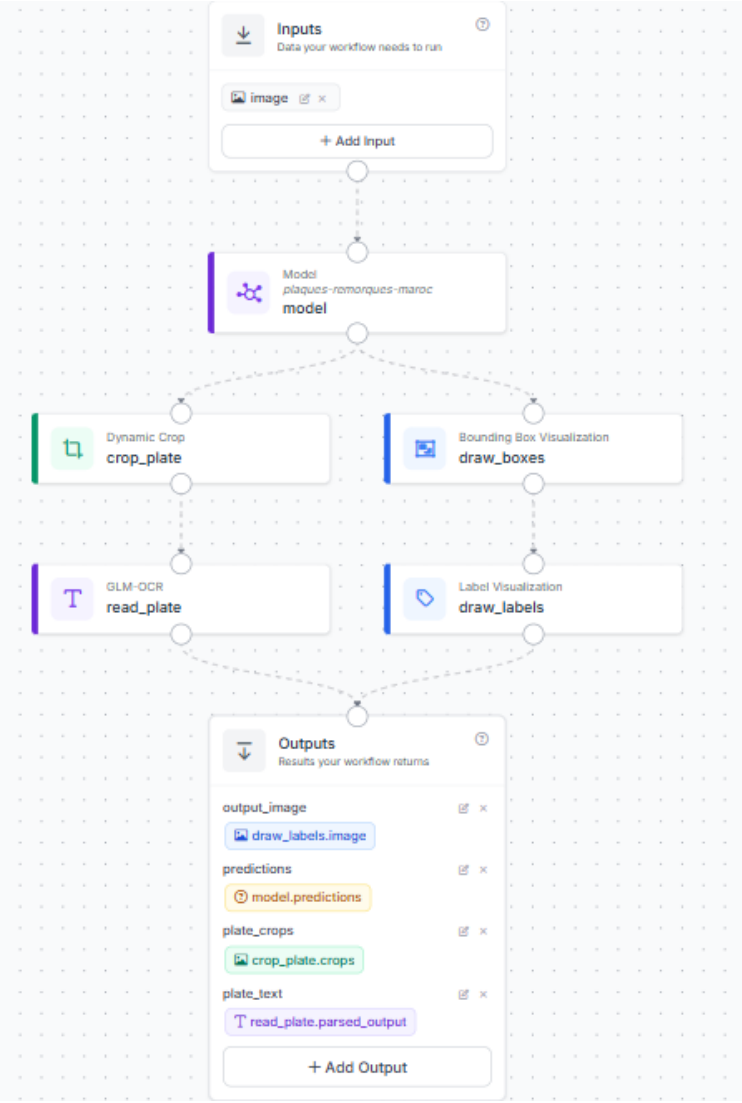


Figure 35 Preview Roboflow pour Remorque

4.7.9 Reconnaissance de la lettre arabe

La zone centrale contient exactement une lettre parmi les lettres autorisées :

أ، ب، ج، د، ه، و، ز، ح، ط، ي، ش

La première reconnaissance est effectuée avec Qwen 3.5 VL 2B, configuré en classification fermée.Cela signifie que le modèle ne peut pas retourner une lettre arbitraire. Il doit choisir parmi la liste autorisée.

Exemple : {

"class_name": "ط",

"confidence": 0.95

}

Cependant, la confiance générée par un modèle visuel n'est pas toujours une preuve suffisante. Sur la plaque : 74467 | 8 | أ , Qwen retournait ي, alors que le recadrage montrait clairement أ.

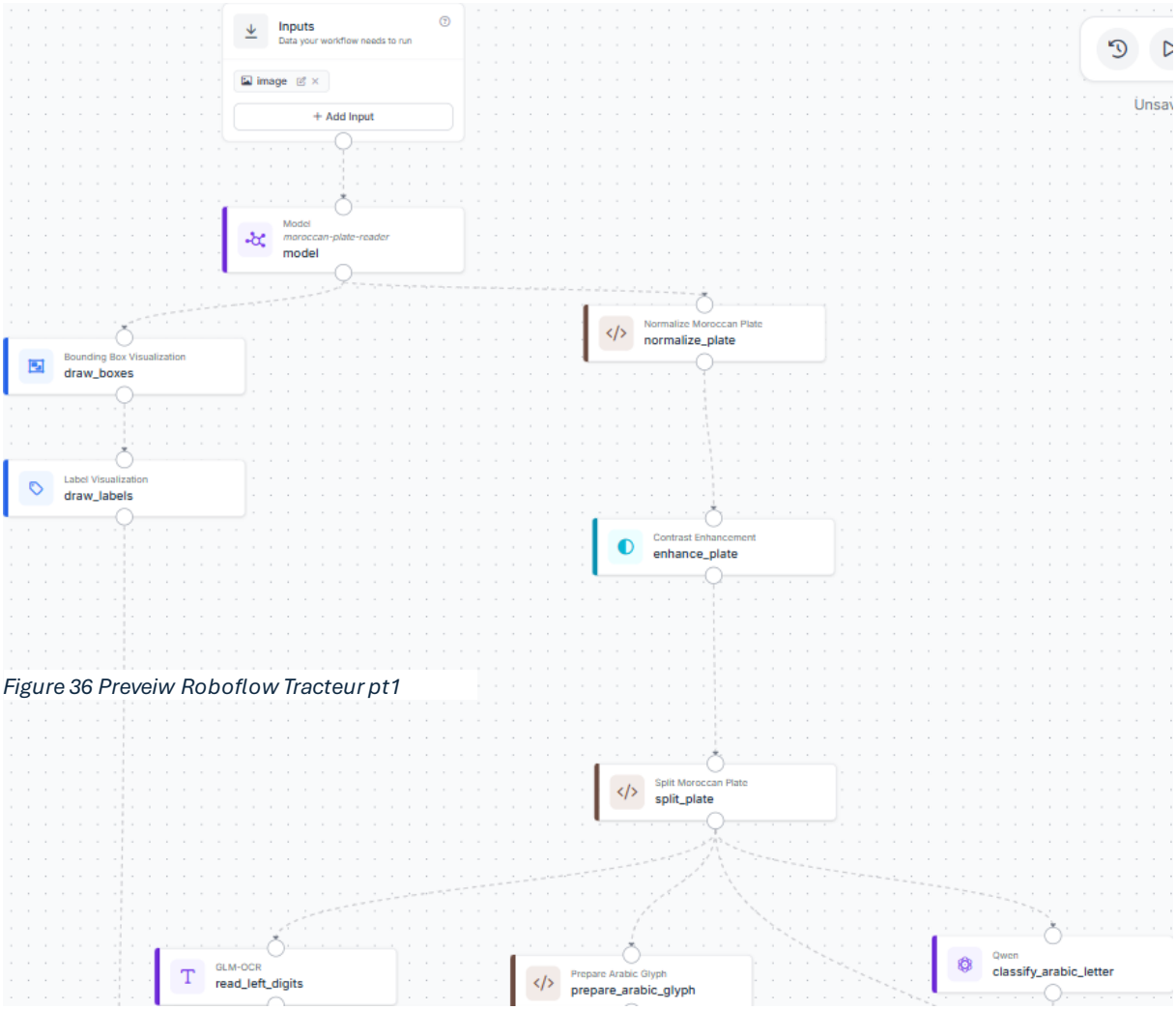


Figure 36 Preveiw Roboflow Tracteur pt1

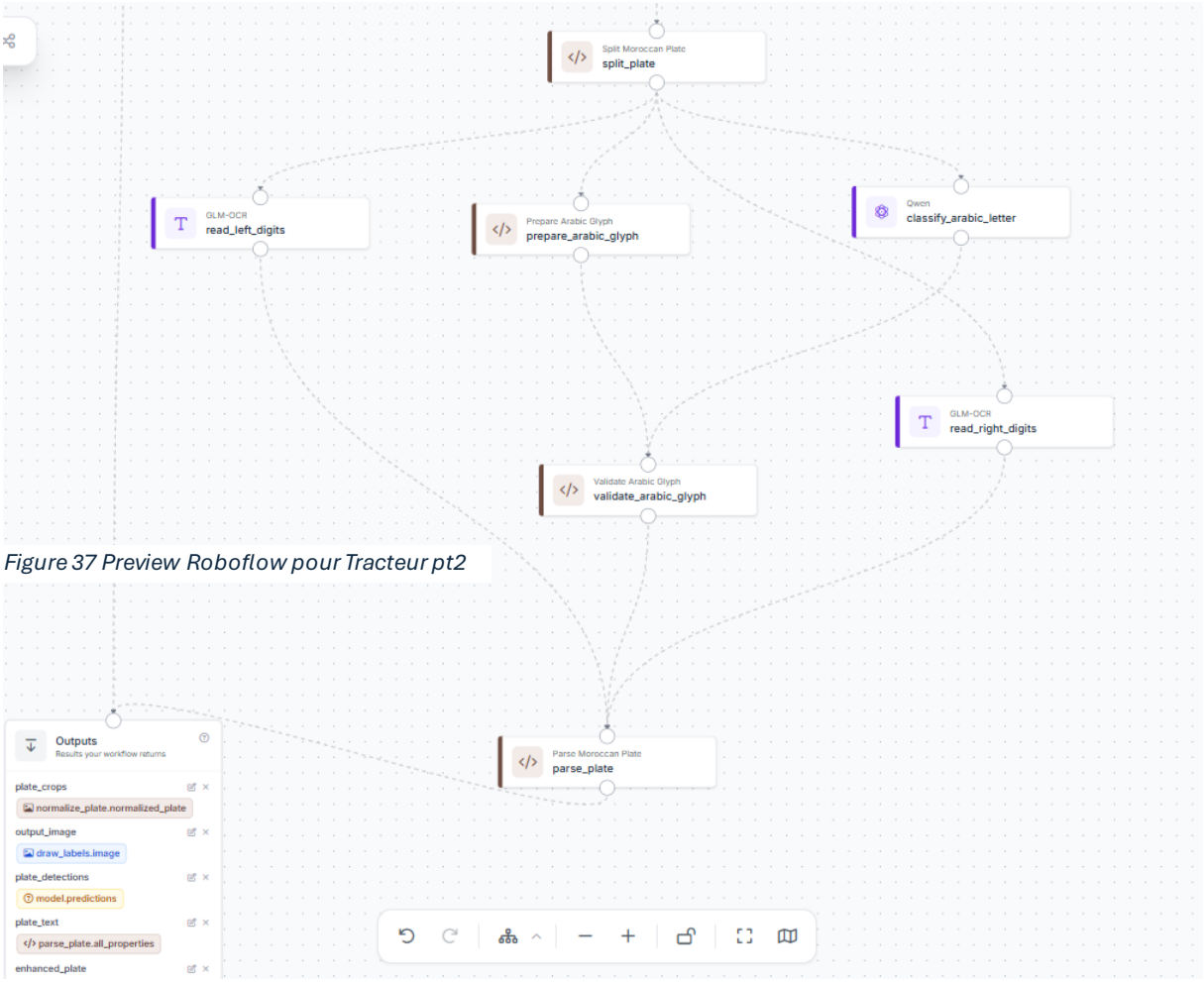


Figure 37 Preview Roboflow pour Tracteur pt2

4.7.10 Problèmes de reconnaissance OCR



Le service OCR peut retourner un résultat incorrect lorsque l'image ne permet pas de lire correctement la plaque. Les corrections possibles sont d'améliorer la qualité de l'image, d'ajuster la position du véhicule, d'utiliser le workflow adapté, de normaliser les caractères, de prévoir une correction manuelle et de vérifier l'existence du véhicule dans la base. L'OCR est donc utilisé comme une aide à l'identification, tandis que la validation finale repose sur les données du système.

Conclusion

Ce chapitre a présenté la réalisation et l'implémentation du système MIZAN. La première partie a décrit l'environnement de développement utilisé, notamment Visual Studio Code, Node.js, npm, PostgreSQL, pgAdmin, les outils de test API, Roboflow et Expo.

L'organisation du projet a ensuite été présentée à travers la séparation entre le backend et le frontend web. Cette organisation permet de distinguer les responsabilités de chaque composant et facilite la maintenance du code.

L'implémentation du backend a permis de développer les fonctionnalités principales du système : authentification, gestion des camions, gestion des chauffeurs, gestion des prestations, contrôle des autorisations, pesée d'entrée et de sortie, calcul du poids net, génération des tickets et communication en temps réel.

L'application web permet aux utilisateurs de superviser les opérations, de consulter les informations et de gérer les différentes entités du système. L'intégration du service OCR facilite l'identification des tracteurs et des remorques à partir de leurs plaques d'immatriculation.

Enfin, les difficultés rencontrées ont permis d'améliorer la fiabilité de la solution grâce à la correction des routes, à la validation des données, à la vérification des autorisations et à l'harmonisation des communications entre les composants.



Chapitre 5

CONCEPTION ET ARCHITECTURE DE L'APPLICATION MOBILE (ÉTUDE A PRIORI)





5.1 Introduction

Après avoir présenté la conception générale du système MIZAN ainsi que la réalisation de la plateforme Web, ce chapitre est consacré à la conception et à l'architecture de l'application mobile PDA.

L'application PDA constitue un complément opérationnel de la solution MIZAN. Elle permet aux utilisateurs autorisés d'effectuer certaines opérations directement depuis un terminal mobile, notamment la consultation des prestations, le scan des QR Codes, le contrôle des autorisations et le suivi des opérations de pesée.

La conception de cette application a été réalisée en tenant compte des contraintes liées à l'utilisation d'un terminal mobile dans un environnement professionnel : simplicité d'utilisation, rapidité d'exécution, communication avec le serveur, sécurité des échanges et gestion des erreurs réseau.

5.2 Objectifs

L'application mobile PDA a pour objectif de faciliter l'exécution des opérations liées aux prestations et aux pesées depuis un terminal mobile.

Les principaux objectifs sont les suivants :

- permettre l'accès aux prestations disponibles ;
- permettre la lecture des QR Codes associés aux tickets ;
- identifier une prestation à partir d'un QR Code ;

  simplifier les opérations réalisées par l'utilisateur sur le terrain ;
- réduire les manipulations manuelles ;

L'application a ainsi été conçue comme une interface mobile complémentaire à l'application Web et non comme un remplacement de celle-ci.

5.3 Contraintes

La conception de l'application mobile a été réalisée en tenant compte de plusieurs contraintes techniques et fonctionnelles :

5.3.1 Contraintes matérielles

L'application doit fonctionner sur un terminal mobile de type PDA équipé d'une caméra permettant notamment de lire les QR Codes.

L'interface doit donc être adaptée à un écran de taille réduite par rapport à un ordinateur.

5.3.2 Contraintes réseau

L'application communique avec le backend MIZAN à travers le réseau local. Elle dépend donc de la disponibilité du serveur backend et de la connectivité réseau du terminal.

Une mauvaise configuration de l'adresse IP du serveur ou une interruption du réseau peut empêcher l'application d'accéder aux services nécessaires.

5.3.3 Contraintes d'utilisation

Les opérations doivent pouvoir être réalisées rapidement et avec un nombre limité de manipulations.

L'interface doit également fournir des indications explicites lorsque :

- un QR Code est invalide ;
- aucune prestation correspondante n'est trouvée ;
- un camion n'est pas autorisé ;
- le serveur n'est pas accessible ;
- une requête échoue ;
- une opération n'est pas autorisée.

5.3.4 Contraintes de sécurité

Les informations échangées entre le PDA et le backend doivent être contrôlées afin d'éviter qu'un utilisateur puisse exécuter une opération non autorisée.

La validation des autorisations est donc réalisée côté serveur et ne repose pas uniquement sur les contrôles de l'application mobile.

5.4 Choix technologiques

Le développement de l'application PDA repose principalement sur les technologies suivantes :

Tableau 5 Tableau de technologie de PDA

Technologie	Utilisation
React Native	Développement de l'application mobile
Expo	Environnement de développement et d'exécution
JavaScript	Langage de programmation
Expo Camera	Accès à la caméra et lecture des QR Codes

Fetch API	Communication avec le backend REST
Socket.IO Client	Communication temps réel
Node.js / Express.js	Backend consommé par l'application
PostgreSQL	Stockage des données
Docker	Hébergement du serveur PostgreSQL

Le choix de React Native permet de développer une application mobile à partir d'une base JavaScript tout en bénéficiant d'une structure adaptée aux applications mobiles.

Expo facilite également les phases de développement, de test et d'exécution sur le terminal.

5.5 Architecture mobile

L'application PDA adopte une architecture client-serveur.

Le PDA constitue la couche cliente et communique avec le backend MIZAN à travers deux mécanismes principaux :

1. des requêtes HTTP vers l'API REST ;
2. une connexion Socket.IO pour les événements temps réel.

L'architecture peut être représentée de manière simplifiée comme suit :

PDA mobile → API REST / Socket.IO → Backend Node.js/Express → Sequelize → PostgreSQL

La base PostgreSQL est hébergée dans un conteneur Docker afin d'assurer une meilleure isolation et une gestion simplifiée de l'environnement de base de données.

L'application mobile ne communique donc pas directement avec la base de données. Toutes les opérations passent par le backend, qui assure la validation, le traitement métier et l'accès aux données.

5.5.1 Communication REST

Les requêtes REST sont utilisées pour les opérations telles que :

- récupération des prestations ;
- recherche d'une prestation à partir d'un QR Code ;
- validation d'une autorisation ;

- récupération des informations nécessaires à l'opération.

5.5.2 Communication temps réel

Socket.IO est utilisé pour transmettre certains événements en temps réel entre le serveur et les applications connectées.

Cette communication permet notamment de recevoir les changements liés aux opérations de pesée sans devoir effectuer continuellement des requêtes de récupération.

5.6 Organisation des écrans

L'application PDA est organisée autour de plusieurs écrans correspondant aux principales étapes du processus.

Écran d'accueil

L'écran d'accueil constitue le point d'entrée de l'application. Il permet notamment d'accéder au scan d'un QR Code et aux fonctionnalités liées aux prestations.

Écran de sélection ou de consultation des prestations

Cet écran permet d'afficher les prestations disponibles pour l'utilisateur PDA.

Les informations affichées peuvent notamment comprendre :

- l'identifiant de la prestation ;
- sa référence ;
- son statut ;
- les informations nécessaires à son traitement.

Écran de scan QR Code

Cet écran utilise la caméra du terminal pour détecter un QR Code.

Après détection, le code est transmis au backend afin d'identifier l'élément associé.

Écran de contrôle de l'autorisation

Une fois le camion identifié, l'application peut demander au backend si celui-ci est autorisé à effectuer l'opération pour la prestation concernée.

Le résultat permet d'autoriser ou de bloquer la poursuite du processus.

Écran d'exécution

Cet écran permet de suivre l'opération sélectionnée et de recevoir les informations nécessaires à son traitement.

L'interface est volontairement orientée vers les informations utiles à l'opérateur afin de limiter les manipulations.

5. 7 Navigation

La navigation de l'application suit une logique progressive.

Le parcours général peut être résumé comme suit :

Accueil → Scan QR / Prestations → Identification → Contrôle d'autorisation → Exécution → Fin de l'opération

Lorsqu'un QR Code est scanné, l'application récupère les informations correspondantes auprès du backend.

Si aucune prestation valide n'est trouvée ou si le camion n'est pas autorisé, le processus est interrompu et un message explicite est affiché.

Cette organisation permet de guider l'utilisateur étape par étape et d'éviter l'accès direct à une opération qui n'aurait pas été validée.

5.8 Maquettes

Les maquettes de l'application PDA ont été conçues autour d'une interface simple et adaptée à une utilisation opérationnelle.

Les principaux écrans prévus sont :

- écran d'accueil ;
- écran de scan QR Code ;
- écran de consultation des prestations ;
- écran de validation d'autorisation ;
- écran d'exécution ;
- écran de confirmation ;
- écran d'erreur.

La conception graphique privilégie une hiérarchie visuelle claire, des boutons suffisamment grands pour une utilisation tactile et des informations synthétiques.

Les couleurs et éléments visuels sont également cohérents avec l'identité graphique générale du projet MIZAN.

5.9 Diagrammes mobiles

Plusieurs diagrammes UML peuvent être utilisés pour représenter le fonctionnement de l'application mobile.

5.9.1 Diagramme de cas d'utilisation

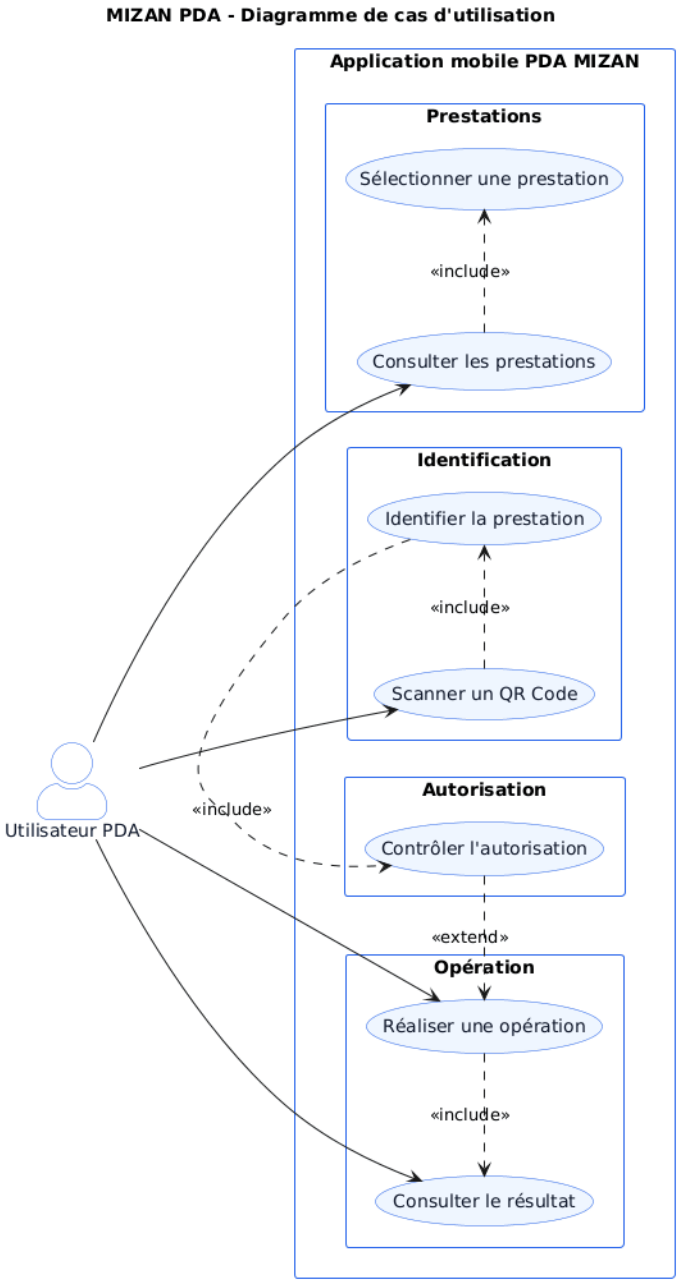


Figure 39 Diagramme Cas d'Utilisation PDA

Le diagramme de cas d'utilisation présente les principales interactions entre l'utilisateur de l'application PDA et le système MIZAN. L'utilisateur peut consulter les prestations

disponibles, scanner un QR Code et identifier la prestation associée. Le système effectue ensuite le contrôle de l'autorisation afin de déterminer si l'opération peut être poursuivie. Lorsque les conditions sont satisfaites, l'utilisateur peut réaliser l'opération et consulter le résultat obtenu. Ce diagramme permet ainsi de définir clairement les fonctionnalités accessibles depuis l'application mobile et le rôle de l'utilisateur dans le processus.

5.9.2 Diagramme de séquence

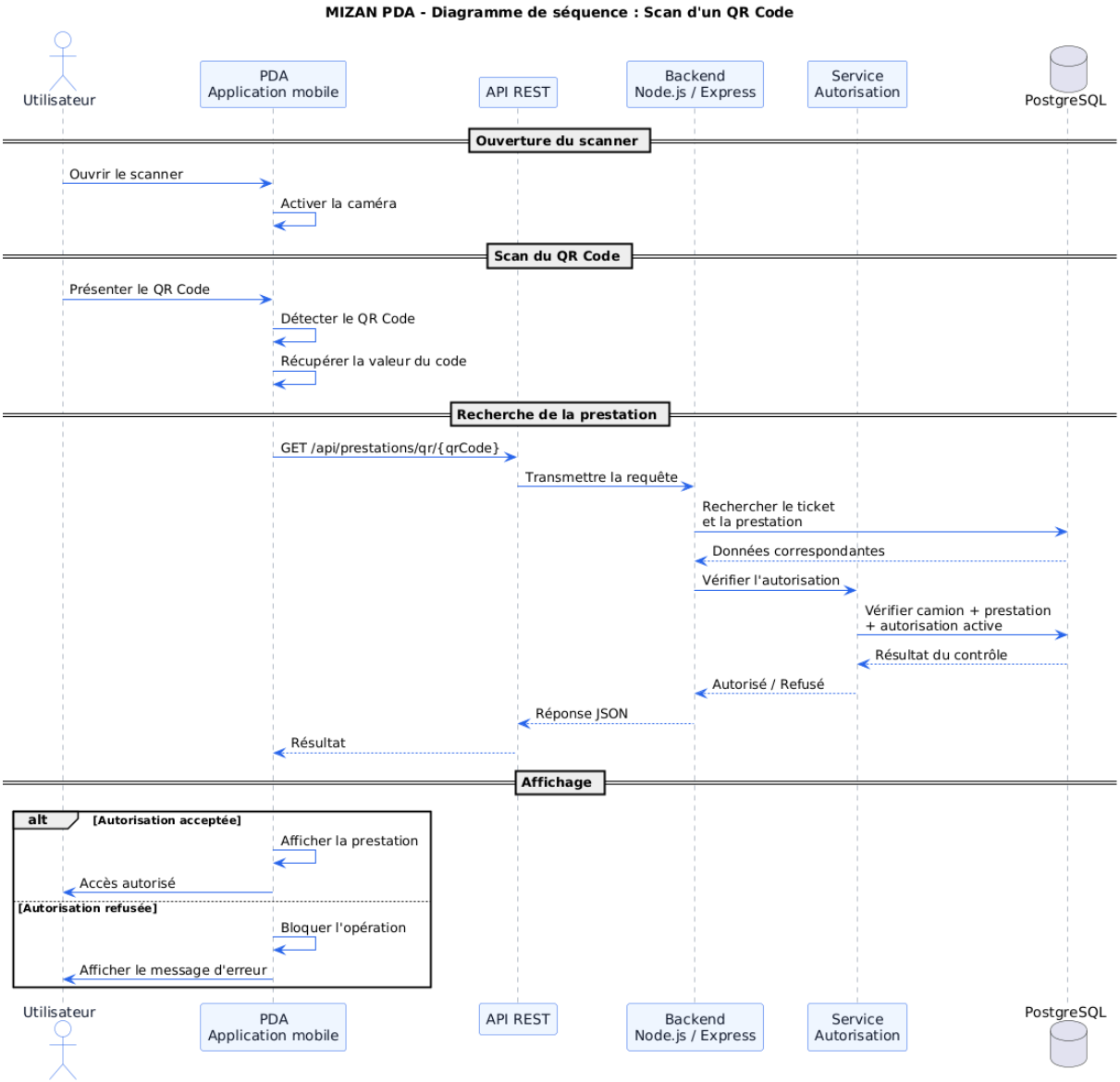


Figure 40 Diagramme de Sequence PDA

Le diagramme de séquence décrit le déroulement technique d'un scan de QR Code depuis l'application PDA. L'utilisateur commence par ouvrir le scanner et présenter le QR Code devant la caméra. Après détection, l'application récupère la valeur du code et envoie une requête à l'API REST. Le backend traite cette requête, recherche les informations correspondantes dans la base de données PostgreSQL et vérifie ensuite les règles d'autorisation. Le résultat du contrôle est finalement retourné à l'application PDA

sous forme de réponse, permettant d'afficher soit l'accès autorisé et les informations de la prestation, soit un message indiquant que l'opération ne peut pas être poursuivie.

5.9.3 Diagramme d'activité

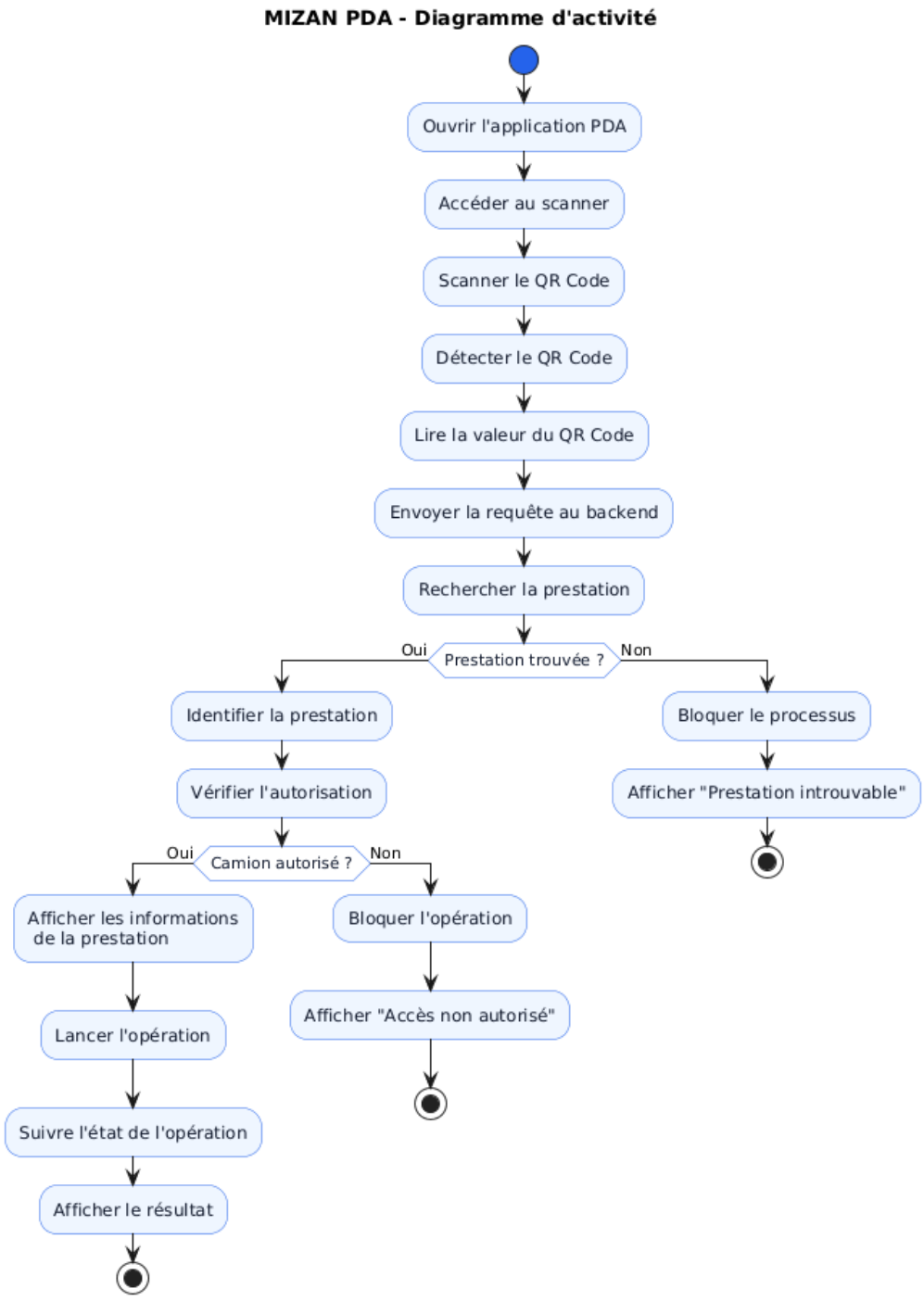


Figure 41 Diagramme d'activité PDA

Le diagramme d'activité représente le déroulement global du processus de traitement d'un QR Code dans l'application PDA. Le processus commence par l'ouverture du scanner et la lecture du QR Code. L'application transmet ensuite les informations au

backend afin de rechercher la prestation correspondante. Une première décision permet de vérifier si la prestation existe. Si elle est trouvée, le système procède au contrôle de l'autorisation du camion. Lorsque l'autorisation est valide, l'opération peut être lancée et son résultat affiché à l'utilisateur. Dans le cas contraire, le processus est interrompu et un message d'erreur est présenté. Ce diagramme permet donc de visualiser les différentes étapes du workflow ainsi que les conditions pouvant entraîner l'arrêt du processus.

5.10 Conclusion

Ce chapitre a présenté la conception et l'architecture de l'application mobile PDA intégrée à la solution MIZAN.

L'application a été conçue comme un complément mobile à la plateforme Web. Elle permet notamment de consulter les prestations, scanner les QR Codes, contrôler les autorisations et interagir avec le backend.

L'architecture repose sur React Native et Expo côté mobile, une API REST et Socket.IO côté communication, ainsi que Node.js, Express.js, Sequelize et PostgreSQL côté serveur.

Cette conception fournit une base adaptée aux opérations mobiles et permet d'intégrer progressivement de nouvelles fonctionnalités. Le chapitre suivant sera consacré aux tests, à la validation et au déploiement de la solution.



Chapitre 6

TESTS, VALIDATION ET DÉPLOIEMENT





6.1 Introduction

Après la conception et la réalisation des différents composants du système MIZAN, une phase de tests est nécessaire afin de vérifier le bon fonctionnement de la solution. Les tests permettent de contrôler les fonctionnalités développées, la communication entre les différents composants, la cohérence des données et le comportement de l'application en cas d'erreur.

La validation porte principalement sur les trois composants de la solution : l'application web, l'application mobile PDA et le backend avec les services associés.

6.2 Stratégie de test

La stratégie de test repose sur plusieurs niveaux complémentaires assurant une couverture complète du système.

Les **tests unitaires et techniques** permettent de vérifier le comportement de fonctions ou de composants particuliers. Ils constituent le premier niveau de validation et assurent que chaque élément fonctionne correctement de manière isolée.

Les **tests d'intégration** permettent de vérifier la communication entre plusieurs composants, par exemple Frontend → API → Backend → Base de données ou PDA → API → Backend → Base de données. Ces tests valident l'interopérabilité des différents modules du système.

Les **tests fonctionnels** permettent de vérifier que les fonctionnalités correspondent aux besoins définis dans le cahier des charges. Ils s'appuient sur les scénarios d'utilisation réels du système de pesage.

Les **tests de validation métier** permettent de vérifier que le processus de pesée respecte les règles métier définies pour MIZAN. Ils assurent que le système répond aux exigences opérationnelles de Marsa Maroc.

6.3 Tests fonctionnels

Les principaux scénarios fonctionnels testés sont présentés dans le tableau suivant.

Tableau 6 Tests Fonctionnels

ID	Fonctionnalité	Résultat attendu
TF01	Authentification	L'utilisateur est authentifié si les informations sont correctes
TF02	Identification camion	Le camion est retrouvé à partir de ses informations

TF03	Identification chauffeur	Le chauffeur correspondant est retrouvé ou enregistré
TF04	Lecture QR Code	Le QR Code est correctement détecté
TF05	Recherche prestation	La prestation correspondante est récupérée
TF06	Contrôle autorisation	L'accès est accepté ou refusé selon les règles définies
TF07	Pesée d'entrée	La pesée est enregistrée avec les informations nécessaires
TF08	Pesée de sortie	La pesée de sortie est enregistrée
TF09	Calcul poids net	Le poids net est calculé à partir des valeurs disponibles
TF10	Génération ticket	Un ticket est généré pour l'opération
TF11	Génération PDF	Le document PDF est accessible
TF12	Historique	Les opérations enregistrées sont consultables

Les tests fonctionnels ont permis de vérifier le parcours global d'une opération de pesée, depuis l'identification jusqu'à l'enregistrement des données. Chaque scénario a été exécuté plusieurs fois dans des conditions variées afin de valider la robustesse du système.

6.3.1 Tests API

Les API REST ont été testées indépendamment de l'interface graphique afin de vérifier le fonctionnement du backend. Cette approche permet d'isoler la logique métier des problèmes potentiels liés à l'interface utilisateur.

Les tests ont notamment porté sur les méthodes HTTP, les paramètres envoyés, les réponses JSON, les codes HTTP, la validation des données, les erreurs et les relations entre les entités.

Un scénario de test peut par exemple suivre le processus suivant : POST /api/pesee/entree → validation des données → création de la pesée → création du ticket → réponse JSON.

Les codes HTTP permettent également de distinguer les différents résultats : 200 ou 201 pour une opération réussie, 400 pour des données incorrectes, 401 pour une authentification nécessaire ou invalide, 403 pour une opération interdite, 404 lorsqu'une ressource n'existe pas, 409 lorsqu'un conflit métier ou de données est détecté et 500 en cas d'erreur interne du serveur.

Ces tests ont permis d'identifier plusieurs erreurs de configuration et de routage au cours du développement, puis de les corriger avant le déploiement.

6.3.2 Tests Web

Les tests de l'application web ont porté sur les principales interfaces fonctionnelles. Les éléments vérifiés sont notamment l'affichage du tableau de bord, l'authentification, la saisie ou détection des matricules, l'identification du chauffeur, la validation du poids, l'enregistrement d'une pesée, la génération du ticket, l'affichage de l'historique, l'actualisation des informations et la réception des événements en temps réel.

Une attention particulière a été accordée à la cohérence des informations affichées entre le frontend et le backend. Les tests ont également vérifié la réactivité de l'interface lors des mises à jour en temps réel via Socket.IO.

6.3.3 Tests PDA

Les tests de l'application PDA ont porté principalement sur le scan des QR Codes, la récupération des prestations et le contrôle des autorisations.

Les scénarios suivants ont notamment été vérifiés : un QR Code valide permet de récupérer les informations associées, un QR Code inconnu affiche un message d'erreur, une prestation inactive bloque l'opération, un camion autorisé accepte le contrôle, un camion non autorisé refuse le contrôle, un backend inaccessible affiche une erreur de communication et une caméra non autorisée informe l'utilisateur du problème.

Ces tests permettent de vérifier que l'application mobile ne poursuit pas une opération lorsque les conditions nécessaires ne sont pas réunies, garantissant ainsi la sécurité et la cohérence du processus.

6.3.4 Tests de performance

Les tests de performance ont pour objectif d'observer le comportement du système lors des opérations courantes. Les éléments étudiés sont notamment le temps de réponse de l'API, le temps de récupération des prestations, le temps de traitement des requêtes de pesée, le temps de génération du ticket, le temps de transmission des événements Socket.IO et le comportement de l'application lors de plusieurs requêtes successives.

Dans le cadre du projet, les tests ont principalement été réalisés dans l'environnement de développement et de démonstration. Ils permettent surtout d'identifier les opérations susceptibles de nécessiter une optimisation lors d'un déploiement à plus grande échelle.

6.8 Validation métier

La validation métier consiste à vérifier que le fonctionnement technique correspond au processus réel de gestion des pesées. Le scénario global peut être résumé comme suit :

identification du véhicule, identification du chauffeur, identification ou sélection de la prestation, contrôle de l'autorisation, acquisition du poids, validation de la pesée, enregistrement dans la base, génération du ticket, suivi de l'opération, réalisation de la pesée de sortie, calcul du poids net et consultation de l'historique.

La validation de ces étapes permet de vérifier la cohérence entre les besoins métier définis au début du projet et les fonctionnalités effectivement développées. Elle assure que le système répond aux exigences opérationnelles de Marsa Maroc.

6.9 Déploiement

Le déploiement de MIZAN repose sur plusieurs composants distincts assurant la séparation des responsabilités et la scalabilité du système.

Le **backend** Node.js/Express constitue le point central de communication entre les applications clientes, les services externes et la base de données. Il orchestre l'ensemble des traitements métier et assure la cohérence des données.

La **base de données** PostgreSQL est exécutée dans un conteneur Docker. Cette approche permet de séparer l'environnement de la base de données du système hôte et de faciliter sa gestion. Le stockage persistant doit être configuré afin d'éviter la perte des données lors du redémarrage du conteneur. L'architecture peut être représentée comme suit : Application Web / PDA → Backend Node.js → Sequelize → PostgreSQL dans Docker.

L'**application web** est exécutée sur le serveur frontend et communique avec le backend via les API REST. Elle constitue l'interface principale de supervision pour les agents et administrateurs.

L'**application PDA** est exécutée sur le terminal mobile et utilise l'adresse réseau du backend pour accéder aux services MIZAN. Elle permet les opérations sur le terrain, notamment le scan des QR Codes et l'identification des chauffeurs.

Le **service OCR** est utilisé pour l'identification automatique des plaques d'immatriculation. La communication avec ce service est réalisée par le backend ou le composant prévu dans le workflow d'identification.

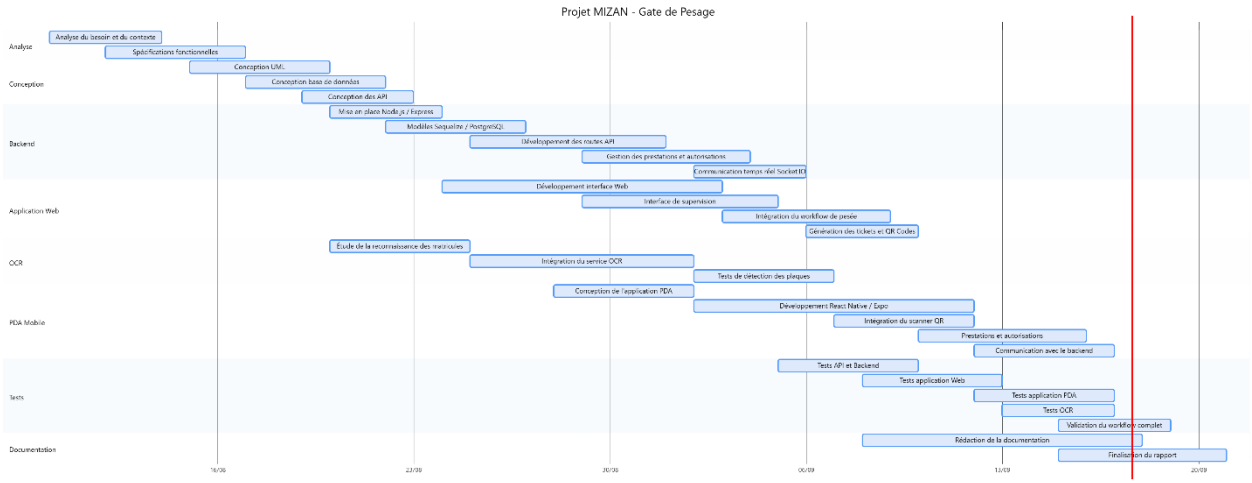


Figure 42 Diagramme de Gantt

6.10 Limites

Malgré les fonctionnalités réalisées, certaines limites subsistent et pourraient faire l'objet d'améliorations futures.

La première concerne la dépendance au réseau local. Une interruption de communication entre le PDA, le backend ou les services externes peut empêcher certaines opérations. Une solution de repli ou un mode hors ligne partiel pourrait être envisagé pour pallier ce problème.

La deuxième concerne le contexte de déploiement. Une partie des tests a été réalisée dans un environnement de développement et de démonstration. Un déploiement industriel nécessiterait des tests supplémentaires dans les conditions réelles d'exploitation, avec un volume de données et un nombre d'utilisateurs plus importants.

Une autre limite concerne l'OCR. La qualité de la reconnaissance peut dépendre de la qualité de l'image, de l'éclairage, de l'orientation du véhicule et de la visibilité de la plaque. Des conditions environnementales défavorables peuvent réduire la précision de la reconnaissance.

Enfin, les performances pourraient être étudiées davantage avec un volume de données et un nombre d'utilisateurs plus importants. Des tests de charge et d'endurance permettraient d'identifier les goulots d'étranglement potentiels et d'optimiser l'architecture en conséquence.

6.11 Conclusion

Ce chapitre a présenté la stratégie de tests mise en place pour vérifier le fonctionnement de MIZAN. Les tests ont couvert les API, l'application web, l'application PDA, les contrôles de sécurité, les performances ainsi que le processus métier de pesée.

Les différentes phases de validation ont permis d'identifier et de corriger plusieurs problèmes techniques liés notamment aux routes API, aux communications réseau, aux autorisations et à l'intégration des différents composants.

Le déploiement repose sur une architecture séparant les applications clientes du backend et de la base de données PostgreSQL, cette dernière étant hébergée dans un conteneur Docker. Cette approche favorise la modularité et la maintenabilité du système.

Le chapitre suivant présente le bilan du projet, les contributions techniques et personnelles ainsi que les perspectives d'évolution.

Conclusion générale

Le projet **MIZAN – Gate de Pesage** a permis de concevoir et de développer une solution informatique destinée à accompagner la digitalisation des opérations de pesée. L'objectif était de proposer un système permettant de centraliser les informations, d'améliorer la traçabilité des opérations et de faciliter le travail des utilisateurs intervenant dans le processus.

La solution réalisée repose sur une architecture composée de plusieurs éléments complémentaires. Une application Web permet la supervision et le suivi des opérations, tandis qu'une application mobile PDA accompagne les opérations réalisées sur le terrain. Ces applications communiquent avec un backend développé avec Node.js et Express.js à travers des API REST. Les données sont stockées dans une base PostgreSQL, déployée dans un conteneur Docker afin de faciliter la gestion de l'environnement de base de données.

Le projet intègre également plusieurs mécanismes permettant d'automatiser certaines étapes du processus. La lecture des QR Codes facilite l'identification des tickets et des prestations, tandis que l'intégration d'un service OCR permet d'exploiter automatiquement les informations issues des plaques d'immatriculation. La communication temps réel avec Socket.IO permet également de transmettre certaines informations entre le serveur et les applications connectées.

La réalisation de MIZAN a nécessité la mise en œuvre de différentes étapes : analyse des besoins, conception UML, conception de la base de données, conception des API, développement du backend, développement de l'interface Web, développement de l'application PDA, intégration des services externes et réalisation des tests.

Les phases de test ont permis de vérifier les principaux scénarios fonctionnels, la communication entre les différents composants et le respect des règles métier. Elles ont également permis d'identifier plusieurs problèmes techniques liés notamment aux communications réseau, aux routes API, aux autorisations et à l'intégration des différents services, puis de mettre en place les corrections nécessaires.

Ce projet a ainsi constitué une expérience concrète dans la conception et le développement d'un système informatique composé de plusieurs applications et services interconnectés. Il a également permis de renforcer les compétences en développement Web et mobile, en conception d'API, en bases de données, en communication temps réel, en intégration de services et en déploiement.

Enfin, la solution développée constitue une base pouvant évoluer en fonction des besoins futurs. Des améliorations peuvent notamment être envisagées concernant le renforcement de la sécurité, l'amélioration de la reconnaissance OCR.

Bibliograaphie et Webographie

- **Google Colab** — environnement utilisé pour l'exécution de notebooks Python et les expérimentations en intelligence artificielle.
  Google Colab
- **Roboflow** — plateforme utilisée pour les travaux liés à la détection et à la reconnaissance des plaques d'immatriculation.
  Roboflow Documentation
- **Sequelize** — ORM utilisé pour la communication entre le backend et PostgreSQL.
  Sequelize Documentation
- **PostgreSQL** — système de gestion de base de données utilisé par MIZAN.
  PostgreSQL Documentation
- **Docker** — technologie utilisée pour exécuter PostgreSQL dans un conteneur.
  Docker Documentation
- **Socket.IO** — bibliothèque utilisée pour la communication en temps réel entre le serveur et les applications.
  Socket.IO Documentation
- **ChatGPT – OpenAI** — utilisé comme outil d'assistance pour la recherche, la compréhension de concepts techniques, le développement et le débogage.
  ChatGPT – OpenAI
- **Claude – Anthropic** — utilisé comme outil d'assistance à la recherche, à l'analyse de code et à la résolution de problèmes techniques.
  Claude – Anthropic
- **Perplexity** — utilisé comme outil de recherche et de consultation de ressources techniques.
  Perplexity
- **YouTube** — utilisé pour consulter des tutoriels et des démonstrations techniques, notamment concernant la détection et la reconnaissance automatique des plaques d'immatriculation avec des outils tels qu'EasyOCR.
  YouTube
- **EasyOCR – GitHub** — bibliothèque de reconnaissance optique de caractères utilisée comme référence pour les expérimentations liées à la lecture automatique des caractères.
  EasyOCR – GitHub